

Meta-Learned Surrogates for Clustering Model Selection

Harry Sevi
Independent Researcher

harry.sevi@protonmail.com

Reviewed on OpenReview: <https://openreview.net/forum?id=5r3W0DcU47>

Abstract

Selecting a clustering without ground-truth labels typically relies on Internal Validity Measures (IVMs) such as Silhouette, Calinski-Harabasz, and Davies-Bouldin. Each IVM encodes a fixed geometric assumption, and no single one aligns with external agreement across heterogeneous datasets. We recast model selection as a supervised prediction problem: METAIVM¹, a meta-learned surrogate that is trained offline on labeled benchmarks and deployed without labels, and that predicts the external quality of an individual (dataset, partition) pair from observable features of partition structure, dataset statistics, and graph topology. Because it scores each candidate partition directly, METAIVM handles algorithm choice, hyperparameter selection, and cluster-count selection in one framework, unlike dataset-level meta-selection methods that only recommend an algorithm. On a benchmark of 223 datasets and 16,889 clustering runs, METAIVM reduces selection regret by 67% relative to the strongest classical baseline (Calinski-Harabasz), and by comparable margins against density-based measures evaluated on the same held-out folds. Controlled comparisons isolate the source of the gain: neither learning from IVM scores alone nor dataset-level meta-selection suffices, the per-partition formulation is what matters, and even linear regression on our features beats every IVM. The learned surrogate adapts its feature reliance to dataset geometry, is robust across four external agreement metrics and across modified candidate pools, and transfers from synthetic to real-world domains, though its accuracy depends on the training distribution. As a preliminary extension to graph community detection, where coordinate-based IVMs do not apply, METAIVM also outperforms modularity-based selection.

1 Introduction

Clustering is one of the most widely used unsupervised learning paradigms, with applications spanning biology, finance, computer vision, and natural language processing. A fundamental challenge in clustering is *model selection*: given a dataset and a set of candidate algorithms with varying hyperparameters, how does one select the best clustering without access to ground-truth labels?

The standard approach relies on *Internal Validity Measures* (IVMs), scores computed from the data and the partition alone, to rank candidate clusterings. Popular IVMs include the Silhouette Coefficient (Rousseeuw, 1987), Calinski-Harabasz Index (Caliński & Harabasz, 1974), and Davies-Bouldin Index (Davies & Bouldin, 1979). The implicit assumption is that these measures correlate with external agreement across datasets, so maximizing them leads to good model selection.

In practice, this assumption often fails. IVMs behave inconsistently across datasets: a measure that identifies the best clustering on one dataset may prefer the worst clustering on another (Simpson et al., 2026; Jeon et al., 2025). On our benchmark, the strongest classical measure, Calinski-Harabasz, attains only $\rho = 0.60$ average per-dataset rank correlation with external agreement (AMI) and a selection regret of 0.217, meaning it selects clusterings that are, on average, 21.7% worse than the best available. Density-based measures

¹Code, data, and a ready-to-use interface: <https://github.com/hsevi/metaivm>.

(DBCV (Moulavi et al., 2014), S_Dbw) are no better under the same protocol, and the Silhouette Coefficient is worse still.

We propose to treat clustering model selection as a *surrogate prediction problem*: rather than relying on hand-crafted validity measures, we learn a surrogate for external clustering agreement from data. Our method, METAIVM, is trained on datasets with known ground-truth labels and deployed without labels. Given a new (dataset, partition) pair, it predicts external agreement (operationalized primarily through Adjusted Mutual Information, AMI) between the partition and the unavailable ground truth, using features derived from the partition structure, dataset statistics, and graph-based measures. This *per-partition* formulation differs from dataset-level meta-selection methods, which recommend algorithm families rather than scoring individual candidate partitions. It also differs from prior per-candidate learned surrogates such as AutoClust (Poulakis et al., 2020) and PoAC (da Silva et al., 2024) through its partition-structural feature space (Section 2). By predicting quality for specific (dataset, partition) pairs, METAIVM naturally handles algorithm choice, hyperparameter selection, and cluster-count selection in a single framework. The framework is model-agnostic: Ridge, MLP, and XGBoost on the same features all substantially outperform classical IVMs.

The key insight is that the evidence predictive of external agreement is context-dependent, varying with data geometry. On spherical clusters, dispersion ratio is the strongest signal; on manifolds, graph conductance dominates; on imbalanced data, cluster size statistics carry most of the information. No single fixed IVM can adapt its evaluation criterion (Appendix K provides a constructive illustration of a specific CH failure mode on anisotropic clusters). A supervised model trained on diverse (dataset, partition, quality) triples learns which signals matter where. Ablations support that the main gains come from the formulation and feature space rather than a specific model: removing IVM features entirely barely changes performance (0.067 vs. 0.071 regret), and even linear regression on our features beats every classical IVM (0.088 vs. 0.217 for the best of them).

Our contributions are:

1. **Per-partition quality prediction.** We formalize clustering model selection as predicting external agreement for individual (dataset, partition) pairs, a more general task than dataset-level meta-selection methods that recommend algorithm families. Prior per-candidate surrogates (AutoClust, PoAC) share this per-partition target but use different feature spaces; we detail the contrast in Section 2. Dataset-level train/test splits prevent information leakage and enable honest evaluation.
2. **Benchmark and evaluation.** We curate a benchmark of 223 datasets (91 OpenML tabular, 123 synthetic, 5 text, 4 image) with 16,889 clustering runs. Five algorithm families produce candidate partitions at benchmark scale (KMeans, GMM, DBSCAN, HDBSCAN, Agglomerative); Spectral Clustering ($O(n^3)$) is added post-hoc on the 189 feasible datasets (Section 6). Evaluation includes paired Wilcoxon significance tests and failure-mode characterization.
3. **Large improvements with principled controls.** Our feature set, combined with any standard ML model, substantially outperforms classical IVMs: Ridge achieves 0.088 regret, XGBoost 0.071, a 59–67% reduction over the strongest classical baseline (Calinski-Harabasz, 0.217). Learned baselines isolate the source: learning IVMs alone does not help, dataset-level meta-selection improves over CH but remains far from per-partition scoring, and partition-level features are essential. Removing IVM features barely affects performance (regret 0.067).
4. **Adaptive feature reliance.** Per-category analysis shows that METAIVM shifts its reliance depending on data geometry: variance heterogeneity on ellipsoidal clusters (81%), graph conductance on manifolds (20%), cluster size statistics on imbalanced data (60%), behavior that no fixed IVM can exhibit. The improvement is concentrated on hard datasets where CH fails (72% regret reduction on the seed-0 hard subset; see Table 5).
5. **Transfer, data efficiency, and robustness.** Synthetic pretraining shows promising transfer to real-world domains (regret 0.100), though transfer is asymmetric. The method outperforms all IVMs with as few as 13 training datasets and remains robust under perturbations to the candidate pool (Section 6). As a preliminary extension, we also apply the same surrogate-learning principle to graph community detection, where coordinate-based IVMs do not apply, obtaining promising results.

2 Related Work

Internal Validity Measures. Classical IVMs evaluate clustering quality without ground truth by measuring properties such as within-cluster compactness and between-cluster separation. The Silhouette Coefficient (Rousseeuw, 1987) measures how similar each point is to its own cluster versus the nearest neighboring cluster. The Calinski-Harabasz Index (Caliński & Harabasz, 1974) computes the ratio of between-cluster to within-cluster variance. The Davies-Bouldin Index (Davies & Bouldin, 1979) averages the worst-case ratio of within-cluster scatter to between-cluster separation. Despite their widespread use, these measures have known limitations: they are biased toward specific cluster shapes (e.g., convex, spherical), do not generalize well across diverse data distributions, and often disagree with each other (Arbelaitz et al., 2013).

Density-Based and Specialized IVMs. Recent work has developed IVMs tailored to specific clustering paradigms. DISCO (Beer et al., 2026) proposes a density-based internal score for clusterings with noise, using density-connectivity distances to handle arbitrary cluster shapes and explicitly evaluating noise label quality, addressing a limitation of classical IVMs that assume all points belong to clusters. While such specialized IVMs improve evaluation within their target paradigm, our approach is orthogonal: rather than designing a better fixed formula for a specific setting, we learn the evaluation function from data across diverse settings.

Adjusted and Cross-Dataset IVMs. Jeon et al. (2025) introduced adjusted IVMs (IVM_A) designed to enable fair cross-dataset comparison. Their approach applies a nonlinear shift transformation and normalizes by the expected score under random label permutation, computed via Monte Carlo simulation over all pairwise class comparisons. While IVM_A addresses the bias of raw IVMs when comparing *across* datasets, we show that for within-dataset model selection, it does not consistently outperform raw Calinski-Harabasz (regret 0.258 vs. 0.217).

IVM Selection and Combination. Simpson et al. (2026) studied the instance space (Smith-Miles & Muñoz, 2023) of clustering validation measures using 18,351 synthetic datasets and meta-features, showing that different IVMs have complementary strengths and proposing a KNN-based selector to choose the best IVM per dataset. Azevedo et al. (2025) explored combining multiple IVMs via multi-objective optimization with NSGA-II. Our approach differs in that rather than selecting among or combining existing IVMs, we learn a surrogate for external agreement that can incorporate IVMs as features alongside additional structural descriptors, and can also operate without IVM inputs entirely.

Meta-Learning for Clustering. Meta-learning has been applied to algorithm selection for clustering (Ferrari & De Castro, 2015; Pimentel & de Carvalho, 2019), where the goal is to predict which *algorithm* will perform best on a given dataset using dataset-level meta-features.

Closer to our per-partition formulation, two prior methods train learned surrogates that map internal validity measures and/or dataset meta-features to external agreement. AutoClust (Poulakis et al., 2020) trains an MLP on a fixed vector of 10 classical IVMs of a candidate clustering to predict Adjusted Rand Index, and uses it as the fitness function inside a Bayesian-optimization loop for hyperparameter tuning; algorithm selection is handled separately by k-nearest neighbors over the same IVM vector. PoAC (da Silva et al., 2024) trains a Random Forest on 37 dataset-level meta-features (Lorena et al., 2019) plus two IVMs (Silhouette, Davies-Bouldin), fit on a meta-database whose training partitions are generated by adding Gaussian noise to ground-truth labels, and uses it as the fitness function in an evolutionary pipeline search.

Both methods are meta-learned per-candidate surrogates for external clustering quality. Compared to them, our approach differs on three concrete design axes.

First, feature space: MetaIVM’s default representation is a 28-dimensional *partition-structural* vector (cluster-size statistics, entropy, imbalance, within/between-cluster variance, centroid separations, cluster densities, and 6 graph-topology features such as conductance and normalized cut), whereas AutoClust uses 10 IVMs of the candidate and PoAC uses 37 dataset meta-features plus 2 IVMs. Two controls probe these alternative feature spaces on our benchmark: a model learned on IVMs alone (Ridge on 3 IVMs, regret 0.337)

and one on IVMs plus dataset descriptors (XGBoost, regret 0.143) both land far above MetaIVM’s 0.071, while removing all IVM features from MetaIVM leaves its regret essentially unchanged (0.067; Table 6). Partition-structural features, not the combination or learning of IVMs, drive the improvement.

Second, training-partition source: our surrogate is trained on 16,889 real algorithm outputs (KMeans, GMM, DBSCAN, HDBSCAN, Agglomerative at real hyperparameter settings) on 223 datasets, matching the deployment distribution. PoAC trains on noise-corrupted ground-truth labels, which is a different learning signal.

Third, scope: we cover 91 real OpenML datasets, 123 synthetic, 5 text, and 4 image datasets, with explicit dataset-level 60/20/20 splits and 5-seed cross-validation; AutoClust reports results on 24 UCI datasets; PoAC on 100 synthetic and 22 UCI datasets. We additionally extend the same formulation to graph community detection (Section 6.8), which neither prior method addresses. Section 6 includes AutoClust-style and PoAC-style baselines evaluated under our protocol as controls.

Beyond these two learned surrogates, our per-partition formulation subsumes algorithm selection: it naturally handles algorithm choice, hyperparameter selection, and cluster-count selection simultaneously, since any partition produced by any method can be scored. Prior dataset-level algorithm-selection work (Ferrari & De Castro, 2015; Pimentel & de Carvalho, 2019) cannot differentiate between KMeans with $k=3$ and $k=10$ on the same dataset, because both are “KMeans.” We can, because we observe the partition itself. This is analogous to the distinction between algorithm selection and performance prediction in supervised learning (Bischl et al., 2016).

Graph Community Detection Evaluation. For graph-structured data, the dominant quality function is modularity (Newman, 2006), which measures the fraction of within-community edges relative to a null model. However, modularity suffers from a well-known resolution limit: it cannot detect communities smaller than a scale that depends on the total number of edges (Fortunato & Barthélemy, 2007). Alternative quality functions include normalized cut, conductance, and surprise (Traag et al., 2015), but no principled method exists for selecting among community detection algorithms on a given graph without ground truth. Our graph extension (Section 6.8) addresses this gap by learning a surrogate quality predictor from graph-topology features.

3 Problem Formulation

3.1 Setup

Let $\mathcal{D} = \{(\mathbf{X}_i, \mathbf{y}_i)\}_{i=1}^N$ be a collection of datasets, where $\mathbf{X}_i \in \mathbb{R}^{n_i \times d_i}$ is the feature matrix and $\mathbf{y}_i \in \{1, \dots, k_i\}^{n_i}$ is the ground-truth class label vector. For each dataset $\mathbf{X}_i$, we generate a set of candidate partitions $\{\pi_{ij}\}_{j=1}^{M_i}$ by running multiple clustering algorithms with varying hyperparameters. The quality of a partition is defined by an *external agreement measure* $Q(\pi, \mathbf{y})$ that compares it to the reference labels:

$$q_{ij} = Q(\pi_{ij}, \mathbf{y}_i). \quad (1)$$

Any standard external measure can play the role of Q (e.g., Adjusted Mutual Information, Adjusted Rand Index, Normalized Mutual Information, or V-measure). We use Adjusted Mutual Information (AMI) as the default throughout, $q_{ij} = \text{AMI}(\pi_{ij}, \mathbf{y}_i) \in [-1, 1]$, and verify that the method is robust to the choice of Q across all four measures above (Table 23).

Our goal is to learn a predictor of external agreement from observable features of the data and the partition, *without* access to $\mathbf{y}_i$ at test time. We factor the predictor through an explicit feature map $\phi : (\mathbf{X}, \pi) \mapsto \phi(\mathbf{X}, \pi) \in \mathbb{R}^p$ that summarises a (dataset, partition) pair as a fixed-length vector, and learn a regression $\hat{q}_{ij} = f(\phi(\mathbf{X}_i, \pi_{ij}))$. The model never sees raw $\mathbf{X}_i$ or π_{ij} directly; it operates on ϕ , whose coordinates factorise into (i) partition-only features (cluster sizes, entropy), (ii) data-only features (dataset statistics, intrinsic dimension), and (iii) partition–data interaction features (within/between-cluster variance, density, graph topology). The full definition of ϕ is given in Section 4.1 and Table 10; the default deployment configuration uses $p = 28$ and an extended one $p = 56$. Deployment is fully unsupervised: the user runs candidate clustering algorithms (with various hyperparameters and cluster counts), the system computes ϕ ,

and the pre-trained f scores each resulting partition. All coordinates of ϕ , including the number of clusters, are extracted from the partition output. This means METAIVM addresses algorithm selection and cluster-count selection in one framework, unlike classical IVMs which are known to be biased toward specific values of k (Arbelaitz et al., 2013).

3.2 Evaluation: Selection Regret

The primary use case is model selection: given a new dataset $\mathbf{X}^*$ and candidates $\{\pi_j^*\}$, select the partition with highest predicted external agreement:

$$j^* = \arg \max_j f(\phi(\mathbf{X}^*, \pi_j^*)). \quad (2)$$

We measure performance via *selection regret*:

$$\text{Regret} = \max_j q_j^* - q_{j^*}^*, \quad (3)$$

which is the gap between the external agreement of the best available partition and the external agreement of the partition selected by our method. Lower regret is better; zero means optimal selection. Note that when multiple partitions achieve the same maximum AMI (common on easy datasets where many algorithms find the ground-truth structure), selecting any of them yields zero regret. We complement regret with Spearman ρ , which measures ranking quality across *all* partitions, not just the top.

3.3 Dataset-Level Splits

To prevent information leakage, we split at the *dataset* level: all partitions from a given dataset appear entirely in train, validation, or test. We use 60/20/20 splits with 5 random seeds. Table 2 reports the pooled mean and standard deviation over the 230 seed–dataset test evaluations; per-seed means are reported separately in Table 14 (Appendix D). This ensures the model is evaluated on datasets it has never seen during training.

3.4 Operative Assumptions and No-Free-Lunch

The formulation above admits a standard No-Free-Lunch obstruction: by the conservation-law argument of Wolpert (1996), averaged over all joint distributions of $(\mathbf{X}, \pi, q)$ triples, every selector (fixed or learned) achieves identical expected regret. Selectors that win consistently in practice rely on structural assumptions about the distribution of problems actually encountered. The empirical gains reported in Section 6 are contingent on three such assumptions: **(A1) Distributional support**: deployment ϕ -values lie in or near the support of the training feature distribution, subsuming both dataset-side coverage (test datasets resemble training datasets in ϕ -space) and partition-side coverage (test candidate partitions resemble those used to train f); **(A2) Learnability/regularity**: $\mathbb{E}[q \mid \phi]$ is approximately learnable from $O(\text{a few hundred})$ labeled datasets with standard regression model classes; **(A3) Feature sufficiency**: ϕ is non-trivially predictive of q , so $\mathbb{E}[q \mid \phi]$ is non-constant. None is directly verifiable on unlabeled deployment data, but each is probed empirically. (A3) is the deepest: Appendix K gives a small constructive illustration of a concrete failure mode of Calinski-Harabasz on two dataset families with anisotropic structure, and Section 6 reports the empirical rate at which the same qualitative failure manifests on our full benchmark (CH exhibits negative Spearman correlation with AMI on 6% of test datasets). Our broader ϕ is empirically sufficient (Table 6). (A2) is the most directly measurable: the learning curve (Table 25, Appendix) exhibits roughly $1/\sqrt{N}$ decay. (A1) is the most practically binding: the cross-domain transfer experiments (Section 6) function as a diagnostic for (A1) and characterise the cost when it fails.

4 Method

4.1 Feature Extraction

We now specify the feature map ϕ introduced in Section 3. For each (dataset, partition) pair, ϕ collects coordinates from four complementary sources:

Partition statistics (12 features): Number of clusters, noise fraction, cluster size statistics (min, max, mean, std, median), size ratio, entropy, Gini impurity, imbalance ratio, singleton fraction.

Dataset descriptors (14 features): Number of samples and features (and their logs), dimensionality ratio, feature-wise mean/std/skewness/kurtosis (averaged), sparsity, intrinsic dimensionality (PCA 95% variance), and pairwise distance statistics (mean, std, median).

Partition-data interaction features (10 features): Within-cluster variance statistics, between-cluster variance, dispersion ratio, cluster density statistics, centroid separation statistics.

Graph-based features (6 features): Using a precomputed k -nearest-neighbor graph, we compute cut fraction, normalized cut proxy, within-cluster edge density (mean and std), and conductance statistics.

IVM features (3 features): Raw Silhouette, Calinski-Harabasz, and (negated) Davies-Bouldin scores are included as features. This allows the model to leverage IVMs as inputs while learning when to trust or override them.

Algorithm encoding (11 features, *optional*): One-hot encoding of the algorithm family (6 categories) and numeric hyperparameters (5 values). These are deployment-optional: the ablation (Table 6) shows removing them has minimal impact (regret 0.071 without algorithm features vs. 0.071 with all 56). Our default configuration excludes them, since they are not always available at deployment time.

In total, we extract 56 features per (dataset, partition) pair. All features are computed without access to ground-truth labels. Our **default deployment configuration** uses the 28 features from the first three groups (partition statistics, partition-data interaction, graph-based), which require only the partition and a precomputed k NN graph. Dataset descriptors, IVM scores, and algorithm encoding are additional features evaluated in ablations (Table 6) but excluded from the default model.

4.2 Model

METAIVM is model-agnostic: any supervised regression model can be used to instantiate f on top of ϕ , mapping the 28 (default) or 56 (extended) feature coordinates to a predicted AMI score. We evaluate three choices of f of increasing complexity:

- **Ridge regression**: L_2 -regularized linear model. Regularization strength α is selected from $\{0.01, 0.1, 1.0, 10.0, 100.0\}$ via validation set performance, then refitted on train+val with the best α .
- **MLP**: ReLU activations, Adam optimizer, features standardized to zero mean and unit variance. Grid search over hidden architectures $\{(256, 128), (512, 256), (256, 128, 64), (512, 256, 128)\}$ and learning rates $\{10^{-3}, 3 \cdot 10^{-3}\}$; each configuration is trained for up to 200 iterations on the dataset-level train fold, with outer arch/lr model selection performed on the dataset-level held-out validation fold. Scikit-learn’s internal early stopping is disabled (`early_stopping=False`) because its default `validation_fraction` splits at the row level and would mix candidate partitions from the same dataset across sklearn’s internal train and validation. Full details in Appendix H.
- **XGBoost** (Chen & Guestrin, 2016): Gradient-boosted regression trees. Hyperparameters are selected via grid search over max depth $\in \{3, 5, 7\}$ and learning rate $\in \{0.01, 0.05, 0.1\}$ (9 combina-

tions), each with up to 500 estimators and early stopping (patience 20) on the validation set. The best configuration is refitted on train+val.

All three models substantially outperform classical IVMs (Table 2), supporting the view that the improvement comes primarily from the formulation and feature design rather than a specific model. XGBoost achieves the lowest regret and is used as the default METAIVM model due to its robustness, native handling of missing values, and interpretability via feature importance.

5 Experimental Setup

5.1 Dataset Collection

We curated a diverse collection of 223 datasets from four sources:

- **OpenML** (91 datasets): Real-world tabular datasets from the OpenML platform (Vanschoren et al., 2014), including the CC18 benchmark suite and additional hand-selected datasets covering biology, finance, physics, and social sciences. Datasets with $>20k$ samples were subsampled; those with >200 features were PCA-reduced to 50 dimensions.
- **Synthetic** (123 datasets): Systematically generated datasets varying dimensionality (2–200), number of clusters (2–10), overlap level, noise fraction, cluster shape (spherical, ellipsoidal, manifold), imbalance ratio, and subspace structure. This ensures coverage of clustering scenarios where different IVMs are known to fail.
- **Text** (5 datasets): 20Newsgroups (Lang, 1995) subsets (all, science, politics, computers, recreation) with TF-IDF features reduced to 50 dimensions via SVD.
- **Image** (4 datasets): MNIST (LeCun et al., 1998) and Fashion-MNIST (Xiao et al., 2017) subsets (2k and 5k samples each) with PCA-50 embeddings.

Full per-source details (subsampling procedures, preprocessing, summary statistics including n, d, k ranges across all 223 datasets) are provided in Appendix E (Table 15). Preprocessing steps (missing-value handling, standardization, k NN graph construction) are documented in Appendix F.

5.2 Clustering Algorithms

For each dataset, we run six candidate algorithm families with varied hyperparameters. Five (KMeans, GMM, DBSCAN, HDBSCAN, Agglomerative) produce candidate partitions at benchmark scale; Spectral-Clustering (below) is skipped at runtime due to its $O(n^3)$ cost and included post-hoc in Section 6 on the 189 feasible datasets:

- **KMeans**: $k \in \{2, 3, 5, 7, 10, 15, 20, 30\}$, $n_init = 10$.
- **Gaussian Mixture**: $k \in \{2, 3, 5, 7, 10, 15, 20\}$, covariance $\in \{\text{full}, \text{diag}\}$.
- **DBSCAN**: $\varepsilon \in \{0.1, 0.3, 0.5, 1.0, 2.0, 5.0\}$, $\text{min_samples} \in \{3, 5, 10, 20\}$.
- **HDBSCAN**: $\text{min_cluster_size} \in \{5, 10, 20, 50\}$, $\text{min_samples} \in \{1, 5, 10\}$.
- **Agglomerative**: $k \in \{2, 3, 5, 7, 10, 15, 20\}$, linkage $\in \{\text{ward}, \text{complete}, \text{average}\}$.
- **Spectral**: $k \in \{2, 3, 5, 7, 10, 15\}$, affinity $\in \{\text{rbf}, \text{nn}\}$. Skipped at benchmark scale due to $O(n^3)$ complexity; included post-hoc in Section 6 on the 189 feasible datasets.

The full hyperparameter grid contains 91 configurations per dataset; after skipping SpectralClustering ($O(n^3)$) and AgglomerativeClustering for datasets with $n > 10,000$ ($O(n^2)$ memory), the effective count ranges from 19 to 79 per dataset (mean: 75.7; lower counts occur for the largest datasets where additional GMM configurations also fail to converge). This yields 16,889 total (dataset, partition) pairs with ground-truth AMI scores. The complete hyperparameter grid is detailed in Appendix C (Table 13). Generalization to unseen algorithm families is examined in Section 6 (candidate-pool robustness, leave-one-algorithm-out, post-hoc Spectral inclusion).

5.3 Baselines

We compare against:

- **Random:** Uniform random selection among candidates.
- **Global best:** The single (algorithm, hyperparameters) configuration with highest average AMI on training data, applied to all test datasets.
- **Classical IVMs:** Silhouette, Calinski-Harabasz, and (negated) Davies-Bouldin used directly for model selection.
- **Adjusted CH** (Jeon et al., 2025): The adjusted Calinski-Harabasz index designed for cross-dataset comparability, computed using the official `btwim` library with 20 Monte Carlo iterations.
- **Ridge regression:** Linear baseline with the same features as XGBoost.
- **MLP:** neural-network baseline with architecture and learning rate selected on the dataset-level validation fold; see Section 4 and Appendix H.

Three additional learned baselines used to isolate the source of improvement (Ridge on 3 IVMs; XGBoost on IVM + dataset descriptors; best-of-three dataset-level meta-selector) are described in Section 6 (Table 4). Implementation details for all training procedures (hyperparameter grids, early stopping, refitting on train+val) are given in Appendix H.

5.4 Metrics

We report five complementary metrics:

- **Selection regret** (primary): Quality gap between the selected and optimal partition.
- **ε -success rate:** Fraction of datasets where regret $< \varepsilon$ (with $\varepsilon = 0.01$).
- **NDCG@5:** Normalized discounted cumulative gain for the top-5 ranked partitions.
- **Spearman ρ :** Rank correlation between predicted and true AMI.
- **Kendall τ :** Concordance probability for pairwise comparisons.

Formal definitions, the dataset-level split protocol (60/20/20 with 5 random seeds, preventing leakage from runs of the same dataset appearing in different folds), and the implementation of all IVM baselines (including the official `btwim` library for Adjusted CH and convergence checks for its Monte Carlo step) are given in Appendix J.

Table 1: Configuration legend. Every named configuration referenced in this paper. “Ridge” / “MLP” / “XGBoost” without a suffix means the default deployment configuration (28-feature `partition_x_graph`). Suffixed variants (e.g., “Ridge-3IVM”) indicate feature-set changes used in controls or ablations.

Name	Feature set (size p)	Model class	Regret
MetaIVM (XGBoost)	<code>partition_x_graph</code> ($p = 28$)	XGBoost, grid on {depth, lr}	0.071
Ridge	<code>partition_x_graph</code> ($p = 28$)	Ridge, α selected on val	0.088
MLP	<code>partition_x_graph</code> ($p = 28$)	MLP, grid on {arch, lr}	0.082
XGBoost-full	<code>all_features</code> ($p = 56$)	XGBoost, same grid	0.071
XGBoost-noIVM	<code>all_features</code> minus IVM ($p = 53$)	XGBoost, same grid	0.067
<i>Controls (Table 4):</i>			
B1 (Ridge-3IVM)	<code>ivm_only</code> ($p = 3$)	Ridge	0.337
B2 (XGB-IVM+DS)	<code>ivm_plus_dataset</code> ($p = 17$)	XGBoost	0.143
B3 (dataset-level)	Best of {5-NN, Ridge, XGB}	dataset-level meta-selector	0.180
AutoClust-style (partial)	4 classical CVIs of candidate*	MLP (60-30-10)	0.195
PoAC-style	115 PyMFE meta-feat + 2 CVIs [†]	Random Forest, 100 trees	0.139

*Partial reproduction: AutoClust’s original design uses 10 CVIs; six (SDBw, CDBw, Tau, Ratkowsky-Lance, McClain-Rao, C-Index) lack maintained Python implementations. See Appendix I.

[†]PoAC’s 37 Lorena 2019 base descriptors expand to 115 columns after PyMFE’s default mean+sd summary statistics; the final matrix used to train PoAC-style is 117 features (115+2 CVIs).

6 Results

Configuration summary and reported numbers. The headline regret is **0.071**: XGBoost on the default $p = 28$ feature map (partition + interaction + graph), under the 5-seed 60/20/20 dataset-level split protocol described in Section 3.3 and Appendix J. Every method reported in Table 2 (fixed IVMs and learned surrogates alike) is scored on the same 46 test datasets per seed (230 seed–dataset test evaluations in total; 158 unique datasets appearing in at least one test fold), so all method-to-method comparisons in the main results are apples-to-apples on identical test observations. Table 1 lists every named configuration referenced in this paper, mapping each to its exact (feature set, model class, training subset, regret). Two additional test-set sizes appear elsewhere in the paper: **158** (leave-one-algorithm-out subset, when one algorithm family is held out at training time) and **189** (post-hoc Spectral inclusion subset, restricted to datasets where SpectralClustering is computationally feasible).

Where multiple numbers appear in this paper for a nominally identical configuration (e.g., “Ridge”), they correspond to different feature sets or averaging scopes; the exact configuration is stated in each table caption and matched to this legend.

6.1 Main Comparison (Experiment 1)

Table 2 presents the main results averaged over 5 dataset-level splits (seeds 0–4). METAIVM (XGBoost) achieves the lowest selection regret across all methods with a regret of 0.071 ± 0.113 (mean $\pm$ std over the 230 seed–dataset test evaluations). The strongest classical baseline on this benchmark is Calinski-Harabasz at 0.217 ± 0.237 ; METAIVM achieves a 67% regret reduction over it, and 72% below Adjusted CH (Jeon et al., 2025) (0.258 ± 0.270). We also evaluate two density-based measures on the same 230 test folds: DBCV (Moulavi et al., 2014) (0.219 ± 0.259), which is designed for density-based clustering and matches our pool’s DBSCAN/HDBSCAN partitions, and S_Dbw (0.334 ± 0.246); both are comparable to but no better than CH, so CH remains the strongest fixed baseline. Improvements are consistent across all metrics.

Figure 1 shows the full distribution of selection regret across datasets. METAIVM achieves consistently low regret, while IVMs exhibit heavy tails with frequent catastrophic failures (regret > 0.5).

A critical observation: even simple linear regression (Ridge) with our features achieves 0.088 regret, already outperforming all classical IVMs (strongest classical baseline: CH at 0.217; density-based DBCV at 0.219 on the same folds) by a wide margin. This shows that the gains are primarily driven by the meta-learning

Table 2: Model selection performance under the 5-seed 60/20/20 dataset-level split protocol (46 test datasets per seed; 230 seed–dataset test evaluations in total; 158 unique datasets appear in at least one test fold). All methods, fixed IVMs and learned surrogates alike, are scored on the same test folds. “ $\pm$ ” reports the standard deviation over the 230 seed–dataset test evaluations, not over the 5 seed means. Seed-level dispersion of METAIVM’s mean regret is small: seed means $\{0.080, 0.075, 0.074, 0.060, 0.064\}$ (matching the per-seed values in Table 14, Appendix D), std 0.008. Learned models use the default 28-feature set. Best in **bold**. MLP training preserves dataset-level train/val separation: sklearn’s internal early-stopping (which randomly splits at the row level) is disabled, and outer architecture/learning-rate selection uses the dataset-level held-out fold (Appendix H).

Method	Regret $\downarrow$	ε -Success $\uparrow$	NDCG@5 $\uparrow$	Spearman ρ $\uparrow$	Kendall τ $\uparrow$
Random	.177 $\pm$.180	.083 $\pm$.276	.467 $\pm$.279	.019 $\pm$.124	.014 $\pm$.087
Global best config	.241 $\pm$.219	.113 $\pm$.317	.554 $\pm$.346	.058 $\pm$.098	.049 $\pm$.082
Silhouette	.342 $\pm$.297	.078 $\pm$.269	.425 $\pm$.349	.433 $\pm$.402	.332 $\pm$.325
Calinski-Harabasz	.217 $\pm$.237	.187 $\pm$.391	.607 $\pm$.311	.604 $\pm$.320	.471 $\pm$.283
Davies-Bouldin	.345 $\pm$.305	.096 $\pm$.295	.405 $\pm$.362	.399 $\pm$.385	.298 $\pm$.313
CH Adjusted (Jeon et al., 2025)	.258 $\pm$.270	.148 $\pm$.356	.518 $\pm$.422	.531 $\pm$.281	.405 $\pm$.237
DBCV (Moulavi et al., 2014)	.219 $\pm$.259	–	–	–	–
S_Dbw (Halkidi et al., 2002)	.334 $\pm$.246	–	–	–	–
Ridge	.088 $\pm$.133	.330 $\pm$.471	.743 $\pm$.314	.662 $\pm$.307	.526 $\pm$.262
MLP	.082 $\pm$.115	.270 $\pm$.445	.738 $\pm$.417	.688 $\pm$.328	.571 $\pm$.304
MetaIVM (XGBoost)	.071 $\pm$.113	.304 $\pm$.461	.779 $\pm$.340	.748 $\pm$.259	.613 $\pm$.247

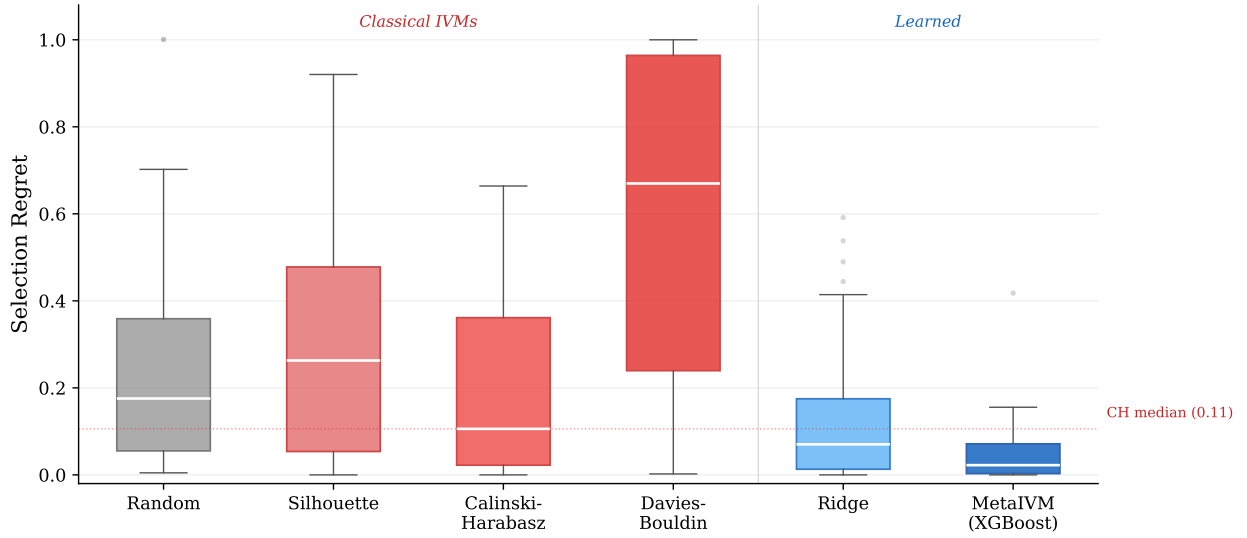


Figure 1: Distribution of selection regret over the 230 seed–dataset test evaluations (5-seed 60/20/20 dataset-level split protocol; 46 test datasets per seed; 158 unique datasets appearing in at least one test fold), using the default 28-feature set (partition statistics + partition-data interaction + graph; no dataset descriptors, IVM, or algorithm features). METAIVM (XGBoost) has the lowest median regret and fewest catastrophic failures. Classical IVMs show high variance with frequent regret > 0.5 .

formulation and feature design. XGBoost provides a further 19% relative improvement over Ridge (0.071 vs. 0.088, $p = 0.022$ by paired Wilcoxon, Table 24) by learning nonlinear feature interactions, but the linear model alone suffices to demonstrate the advantage of our approach over all IVM baselines.

Table 3: Performance stratified by dataset source. N is the total number of (seed, dataset) test evaluations pooled across the 5-seed 60/20/20 protocol; the totals sum to 230 across all slices. Learned models use the default 28-feature METAIVM configuration. Regret is the mean over pooled evaluations, matching the source-stratified rows in Table 7 exactly. METAIVM outperforms classical IVMs on every slice; real-data performance is comparable to synthetic.

Slice	N	MetaIVM regret $\downarrow$	CH regret	Silhouette regret
All	230	.071	.217	.342
Synthetic only	122	.068	.210	.406
Real only (OpenML+text+image)	108	.076	.226	.286
OpenML only	100	.079	.207	.270
Text only	5	.063	.223	.250
Image only	3	.021	.563	.594

Table 4: Learned baselines isolating the source of improvement (5 seeds). Learning alone (B1) does not help; dataset-level meta-selection (B3) improves over CH but remains far from per-partition scoring; partition-level features are essential.

Method	d	Regret $\downarrow$	Spearman $\rho \uparrow$
<i>Fixed IVMs (no learning)</i>			
Raw Silhouette	–	.342	.433
Raw Calinski-Harabasz	–	.217	.604
<i>Learned baselines: prior work reproduced on our benchmark[†]</i>			
AutoClust-style (Poulakis et al., 2020)	4	.195 $\pm$.028	–
PoAC-style (da Silva et al., 2024)	117	.139 $\pm$.025	–
<i>Learned baselines: controls isolating the source of improvement</i>			
B1: Ridge on 3 IVMs (AutoClust-simplified)	3	.337	.391
B2: XGB on IVM + dataset desc. (PoAC-simplified)	17	.143	.666
B3: Dataset-level meta-selector [†]	14	.180	–
<i>METAIVM (per-partition features)</i>			
Ridge (partition_x_graph , 28 features, no IVM)	28	.088	.662
MetaIVM XGB (partition_x_graph, 28 feat, no IVM)	28	.071	.748

[†]Best of 5-NN, Ridge, and XGBoost on same descriptors (Ridge selected).

[‡]Both are our reproductions (no public code released with either paper); see Appendix I for details.

Performance by dataset source. Because our benchmark is synthetic-heavy (123/223 datasets), a natural concern is whether the aggregate result is driven by synthetic data. Table 3 reports performance stratified by dataset source, pooled across the 230 seed–dataset test evaluations. METAIVM outperforms classical IVMs on every slice: on real-only data (regret 0.076 vs. CH 0.226) and on synthetic-only data (0.068 vs. CH 0.210) the improvements are similar in magnitude, and analogous patterns hold for OpenML, text, and image subsets individually. The aggregate result is not driven by synthetic data alone.

6.2 Isolating the Source of Improvement

To determine whether the gain comes from *learning per se*, the *per-partition formulation*, or the *feature space*, we compare against three principled learned baselines (Table 4).

These baselines are designed as controls to isolate the source of improvement, not as exhaustive alternatives. Four findings emerge:

Table 5: Conditional performance by CH difficulty on a single representative test fold (seed 0, $N = 46$ datasets, default 28-feature XGBoost). Stratification is by per-dataset CH regret computed on that fold. This table is an illustrative slice of the aggregate result in Table 2, not a re-aggregation across all 230 seed-dataset pairs; the numerical values are single-seed and should be read as showing *where* the improvement is concentrated rather than as full-benchmark uncertainty estimates. METAIVM provides the largest gains on hard datasets (CH regret > 0.3), winning 13 out of 14 cases.

Regime	N	CH regret	MetaIVM regret	Gain
Easy ($\text{CH} \leq 0.05$)	19	.015	.050	−.035
Medium ($0.05 < \text{CH} \leq 0.3$)	13	.114	.057	+.058
Hard ($\text{CH} > 0.3$)	14	.514	.142	+.372
All	46	.195	.080	+.115

1. **Learning IVMs alone does not help** (B1: regret $0.337 \approx$ raw Silhouette). The three classical IVM scores do not contain enough complementary signal for a learned combination to meaningfully improve selection.
2. **Dataset-level meta-selection helps but is insufficient** (B3: 0.180 vs. strongest classical baseline CH 0.217). The best dataset-level meta-selector (Ridge on 14 dataset descriptors, selected from 5-NN/Ridge/XGBoost via validation) improves over every classical IVM, showing that dataset-level learning is useful. However, it remains far worse than per-partition METAIVM (0.071), indicating that dataset-level descriptors alone are insufficient and that observing the candidate partition is critical.
3. **Adding dataset context helps but plateaus** (B2: 0.143). IVM scores combined with dataset descriptors improve over raw IVMs, but without partition-level features the model cannot distinguish between different clusterings of the same dataset.
4. **Prior learned-surrogate methods reproduced on our benchmark** confirm the same pattern. Since neither AutoClust nor PoAC released code, we reimplemented both from their paper descriptions (Appendix I); we denote these reimplementations *AutoClust-style* and *PoAC-style*. AutoClust-style (MLP on classical CVIs of the candidate) achieves regret 0.195 ± 0.028 , a modest improvement over the best single classical IVM (CH, 0.217): learned regression over a handful of CVIs helps only marginally. PoAC-style (Random Forest on 115 PyMFE columns + 2 CVIs; 117 features total) does better at 0.139 ± 0.025 , showing that dataset-level meta-features contribute additional signal, but still substantially worse than per-partition METAIVM (0.071). The progression CH 0.217 $\rightarrow$ AutoClust-style 0.195 $\rightarrow$ PoAC-style 0.139 $\rightarrow$ METAIVM 0.071 is consistent with a substantial contribution from partition-structural features (supported more directly by the matched B1/B2 controls and feature ablations); PoAC and METAIVM also differ in model class and reproduction protocol, so this progression is corroborating rather than causally isolating.

Where does the improvement come from? The aggregate gains above are concentrated on datasets where classical IVMs fail. Stratifying test datasets by CH difficulty:

On **easy datasets** where CH already succeeds (regret ≤ 0.05), METAIVM is slightly worse; it over-refines simple cases where the best partition is obvious. On **hard datasets** where CH fails badly (regret > 0.3), METAIVM reduces regret from 0.514 to 0.142, a 72% improvement, winning 13 out of 14 cases. This is where the method’s value is concentrated: precisely the datasets where classical IVMs provide no useful guidance. A natural follow-up question is whether a hybrid strategy (trusting CH when it is reliable and METAIVM otherwise) could combine their strengths. The challenge is that identifying CH’s reliability regime *without labels* is itself the core problem; METAIVM is learning exactly that decision boundary from observable partition and dataset statistics.

Table 6: Feature ablation (XGBoost, 5 seeds). d = number of features. “All features” includes IVM + algorithm encoding (56 total). Differences from Table 2 reflect different feature sets.

Feature Set	d	Regret $\downarrow$	Spearman $\rho \uparrow$
Dataset descriptors only	14	.303	–
IVM only	3	.188	.614
Graph only	6	.096	.700
Partition only*	22	.082	.747
Partition + dataset	36	.078	.746
No IVM, no algorithm	42	.068	.777
No IVM features	53	.067	.757
No algorithm features	45	.071	.745
All features	56	.071	.742
Compact: top-15 features	15	.092	.749
Compact: top-8 features	8	.094	.737

*Includes partition statistics (12) + partition-data interaction features (10).

6.3 Candidate-Pool Robustness

Since regret is defined relative to the candidate pool, a natural concern is whether METAIVM is specialized to the specific menu of algorithms tested. To assess this, we train METAIVM on the full pool and evaluate on systematically modified pools at test time. METAIVM outperforms CH across *all 11 pool configurations tested*, including: dropping each individual algorithm family (regret 0.048–0.059 vs. CH 0.134–0.205), restricting to KMeans-only candidates (0.036 vs. 0.085), restricting to density-based candidates only (0.023 vs. 0.119), random 50% subsampling per dataset (0.055 vs. 0.174), and filtering by cluster count $k \leq 3$ or $k \geq 5$ (0.042–0.054 vs. 0.127–0.139). Notably, the model trained on the full pool generalizes to pools that contain only a single algorithm family at test time. The full breakdown is in Table 28 (Appendix). This supports the view that the learned surrogate captures a general ranking principle over partition quality, not a pool-specific selection rule.

6.4 Feature Ablation (Experiment 2)

Table 6 shows the impact of different feature groups. The key findings are:

1. **IVMs as features are useful but not necessary:** Removing all 3 IVM features barely affects performance (regret 0.067 without IVM vs. 0.071 with all 56 features). The model without any IVM or algorithm input (42 features, regret 0.068) still dramatically outperforms all classical IVMs (strongest classical baseline: CH at 0.217). This directly addresses the concern that METAIVM is “just a learned correction layer on top of CH.”
2. **Partition statistics alone are powerful** (regret 0.074, 22 features), outperforming all classical IVMs. Simple features like cluster entropy, Gini impurity, and size ratios carry substantial signal.
3. **Graph features alone** (6 features, regret 0.100) already outperform all IVMs, because k NN graph conductance captures cluster separability robustly across shapes.
4. **A compact 15-feature model** (selected as the 15 most important features, stable across ≥ 3 of 5 training seeds) achieves 0.092 regret, still far better than any IVM. Eight features stable across *all* seeds achieve 0.094 regret.

6.5 Robustness to Algorithm Diversity (Experiment 3)

To test whether the model depends on seeing all algorithm families during training, we perform leave-one-algorithm-out evaluation (Table 19 in Appendix). Removing DBSCAN has the largest impact ($\Delta = +0.023$

regret), consistent with its unique ability to discover non-convex clusters. All other algorithms have minimal impact ($\Delta < 0.007$), indicating robust generalization.

Leave-one-generator-family-out. Random 60/20/20 splits may overestimate cross-family transfer because synthetic datasets from closely-related generators appear in both train and test folds. To probe this we hold out entire synthetic generator families and retrain METAIVM on the remaining 211-220 datasets. Across the 12 synthetic families with at least 3 datasets, mean regret is 0.145 (vs. 0.071 under random CV), maximum 0.487. Most families transfer well: **grid** 0.025, **aniso** 0.026, **ellipsoidal** 0.049, **elongated** 0.050, **circles** 0.056, **noisy** 0.059, **varying** 0.075, **spirals** 0.093, **blobs** 0.094. Three families degrade substantially: **moons** 0.358 (only 4 datasets; no manifold-arc analog remains), **subspace** 0.375 (specific structural pattern), **imb** 0.487 (holding out all imbalanced datasets removes the training signal for imbalance-adaptive scoring). The pattern is consistent with our operative assumption A1 (Section 3.4): transfer succeeds when the held-out family shares ϕ -space with the remaining training pool and degrades when it does not. Full breakdown in Appendix (`logfo_results.csv`).

6.6 Cross-Domain Transfer (Experiment 6)

We frame these experiments as a diagnostic for the distributional-support assumption (A1) introduced in Section 3.4. Can a model trained on one type of dataset generalize to another? Training exclusively on synthetic point-cloud datasets (blobs, ellipsoids, manifolds) and testing on real-world tabular data yields 0.100 regret vs. CH’s 0.235, with $\rho = 0.887$ on unseen image and text datasets (vs. CH’s 0.722). Transfer is asymmetric: real $\rightarrow$ synthetic is weaker (0.238) and underperforms raw CH on synthetic targets (0.193), suggesting synthetic data provides broader ϕ -space coverage. Read through the lens of (A1), the 0.071 (in-distribution) vs. 0.100 (mild shift) vs. 0.238 (severe shift) progression quantifies the cost of moving outside the training feature support. Full breakdown in Table 18 (Appendix).

6.7 Additional Analyses

On the representative seed-0 test fold, the predicted-versus-true AMI scatter (Figure 3, Appendix) shows strong agreement ($R^2 = 0.810$; the figure visualizes a single fold and full 5-seed aggregates are in Table 20). Over the full 230 seed-dataset test evaluations, METAIVM achieves per-dataset Spearman $\rho = 0.748 \pm 0.259$, compared to $\rho = 0.604 \pm 0.320$ for CH (the strongest of the three standard compactness-separation IVMs by ρ ; Table 20, Appendix). When all 56 features are used, Calinski-Harabasz is the top feature (35% importance), yet removing it barely hurts (Table 6): the model appears to route around CH via redundant structural features when CH is misleading. A compact 15-feature model achieves 0.092 regret, still far better than any IVM. Per-category analysis (Table 8) shows that METAIVM adapts its reliance across geometries: variance heterogeneity dominates on ellipsoidal clusters (81%), conductance on manifolds (20%), cluster size statistics on imbalanced data (60%). This adaptive behavior, which no fixed IVM can exhibit, is consistent with the observed performance gap.

Controlled shifts along dataset properties. To characterize where the improvement is uniform vs. concentrated, we bin the 5-seed per-dataset regrets along five axes: number of classes k , feature dimensionality d , dataset size n , class imbalance ratio, and source (Table 7). Mean regret is essentially flat across $k \in [2, 21+]$ (range 0.039–0.083) and across imbalance regimes (0.035–0.099). Dimensionality shows a mild trend: $d \in [6, 15]$ is best at 0.047, $d > 50$ worst at 0.104. Dataset size shows a similar mild trend: $n < 3000$ averages 0.070, $n > 3000$ averages 0.093. OpenML and synthetic sources achieve comparable regrets on pooled held-out evaluations (0.079 vs. 0.068), with real overall (OpenML + text + image, $N = 108$) at 0.076; the aggregate result is not synthetic-driven.

The improvement over Silhouette, Calinski-Harabasz, and Davies-Bouldin is highly significant (paired Wilcoxon $p < 10^{-12}$; Table 24, Appendix). We do not report a p -value against DBCV or S_Dbw because these were added post-hoc after the Wilcoxon analysis was run. The method is robust across four external metrics: AMI, ARI, NMI, and V-measure all show consistent rankings (Table 23, Appendix). With just 13 training datasets (10%), METAIVM already achieves 0.116 regret, better than all IVMs (Table 25,

Table 7: Controlled shifts: METAIVM regret across 5-seed per-dataset evaluations, binned by dataset property. N counts (seed, dataset) pairs in each bin. Regret is roughly flat across k and imbalance; mild dependence on d and n at the extremes; real data (OpenML) is comparable to synthetic.

Axis	Bin	N	Regret (mean $\pm$ std)
n_{classes}	$k = 2$	58	.083 $\pm$.119
	$k \in [3, 5]$	85	.081 $\pm$.122
	$k \in [6, 10]$	69	.059 $\pm$.072
	$k \in [11, 20]$	14	.039 $\pm$.076
	$k \geq 21$	4	.063 $\pm$.089
Dimensionality	$d \leq 5$	90	.083 $\pm$.129
	$d \in [6, 15]$	48	.047 $\pm$.064
	$d \in [16, 50]$	72	.066 $\pm$.081
	$d > 50$	20	.104 $\pm$.135
Dataset size n	$500 \leq n < 1500$	117	.070 $\pm$.114
	$1500 \leq n < 3000$	64	.060 $\pm$.088
	$n > 3000$	49	.093 $\pm$.103
Imbalance ratio	Balanced (< 1.5)	158	.071 $\pm$.113
	Mild (1.5–3)	24	.086 $\pm$.073
	Moderate (3–10)	24	.035 $\pm$.025
	Extreme (> 10)	24	.099 $\pm$.125
Source	Synthetic	122	.068 $\pm$.120
	OpenML	100	.079 $\pm$.088
	Text	5	.063 $\pm$.082
	Image	3	.021 $\pm$.029

Table 8: Dominant features shift across dataset geometries; the model adapts its reliance to data structure.

Dataset type	Dominant feature (importance)	Why it matters
Spherical clusters	Entropy (0.54)	Balanced cluster sizes signal correct k
Ellipsoidal clusters	Within-var std (0.81)	Heterogeneous variance reveals shape mismatch
Manifold clusters	Conductance (0.20)	Graph structure captures non-convex geometry
Imbalanced clusters	Cluster size std (0.60)	Correct partitions preserve the imbalance pattern
Real-world data	Entropy (0.17), centroid sep (0.11)	No single signal dominates

Appendix). Zero-engineering approaches (graph spectral embeddings, GNN) also outperform all IVMs (Appendix G), suggesting the formulation is robust to feature design choices.

Failure analysis (Table 26, Appendix) reveals that the model struggles on datasets with high noise, subspace structure, or extreme imbalance; 19% of test datasets have regret > 0.15 . Text and image domains have zero high-regret cases.

6.8 Extension: Graph Community Detection

Classical IVMs require point coordinates and **cannot be applied** to graph community detection. METAIVM is naturally applicable because our graph-only features (conductance, edge density, normalized cut proxy) require only the adjacency structure and a partition. We evaluate on 80 benchmark graphs (SBM, 2–10 communities, 5 algorithms at multiple resolutions; Table 9).

METAIVM achieves 0.018 regret ($\rho = 0.946$), a 61% reduction over modularity, while the edge-cut fraction and conductance are both anti-correlated with external agreement ($\rho < 0$). On a preliminary set of 7 real-

Table 9: Community detection model selection on 80 benchmark graphs (2,000 runs, 5 seeds). Classical IVMs (Silhouette, CH, DB) cannot be computed because no point coordinates exist. METAIVM outperforms all graph-based heuristics.

Method	Regret $\downarrow$	Spearman $\rho \uparrow$
<i>Not applicable (require point coordinates)</i>		
Silhouette / CH / DB	n/a	n/a
<i>Graph-based heuristics</i>		
Random	.259 $\pm$.318	—
Edge-cut fraction*	.677 $\pm$.341	−.259 $\pm$.489
Conductance	.655 $\pm$.357	−.226 $\pm$.506
Modularity	.045 $\pm$.088	.872 $\pm$.205
MetaIVM (graph features)	.018 $\pm$.044	.946 $\pm$.107
Regret uses NMI as the external agreement metric (no point coordinates, so AMI’s permutation model is unavailable for some graph baselines). *The edge-cut fraction (between-cluster edges / total edges) coincides with 1−coverage; as a selection rule it is identical to a coverage-maximizing selector, so we report the two as a single baseline.		

world attributed graphs (Table 21, Appendix), a leave-one-graph-out evaluation places METAIVM below modularity (0.004 vs. 0.028 regret); with only 7 graphs this is a high-variance, preliminary indication rather than a headline result. Together, these experiments suggest that the surrogate-learning principle extends to settings where classical IVMs cannot operate.

7 Discussion

Why do IVMs fail, and why does MetaIVM succeed? Classical IVMs measure fixed geometric properties that correlate with external agreement only when clusters match geometric assumptions. Our analysis (Table 8) reveals that the predictive signal shifts across regimes, and no fixed IVM can adapt. Ablations support that gains come from the formulation and feature space: Ridge alone beats all IVMs, removing IVM features barely hurts, and learned baselines (Table 4) show neither learning alone nor dataset-level selection suffice. The model uses CH when informative and routes around it when misleading.

Scope and assumptions. Our goal is not universal clustering quality but a surrogate for external agreement in settings where benchmark labels define the reference partition offline. This is a supervised surrogate-learning setup for unsupervised deployment (not circular, but bounded): the method inherits the notion of quality used in training. Multi-target analysis (Table 23, Appendix) shows robustness across AMI, ARI, NMI, and V-measure, but we do not claim robustness to arbitrary task-specific criteria. Pointwise regression and ranking losses achieve similar regret (Table 22, Appendix), suggesting the formulation matters more than the loss.

When are classification labels an appropriate clustering target? Our benchmarks treat each dataset’s classification labels as the reference partition for AMI. This is an operational choice: classification labels provide an unambiguous, reproducible ground truth and are the same target used by most prior meta-learning work on clustering. In some real-world applications, however, the “natural” clustering may differ from the labeled classification: customers can be grouped by demographics, purchase behavior, lifetime value, or risk profile, all valid, all producing different partitions. METAIVM is a framework, not a specific policy about what counts as a good clustering: the same infrastructure would work with hierarchical labels, application-specific tags, or user-defined similarity signals, provided a labeled meta-benchmark exists. Users deploying METAIVM with a specific application clustering goal in mind should retrain on labeled examples that reflect that goal; where the deployment target matches the benchmark target (classification-label recovery on tabular / image / text data), our results are directly applicable. This limitation is inherent to

any supervised surrogate for unsupervised evaluation and is shared with AutoClust (Poulakis et al., 2020), PoAC (da Silva et al., 2024), and dataset-level meta-selection work.

Label semantics. We do not claim that benchmark class labels define the uniquely correct clustering of a dataset. METAIVM learns to predict agreement with the reference partitions used in training. This is appropriate when users want clusters aligned with semantic labels similar to those in the training benchmark (e.g., disease subtypes, object classes, curated taxonomies). It is less appropriate when the desired clustering is task-specific, multi-resolution, or orthogonal to class labels. In such cases, the relevant notion of quality must be encoded in the training distribution.

Benchmark dependence and generalization. Our benchmark is synthetic-heavy (123/223 datasets) by design for controlled geometry coverage, not as a natural-frequency sample. The transfer asymmetry (synthetic→real: 0.100, real→synthetic: 0.238) confirms distribution dependence. On pooled held-out OpenML evaluations, METAIVM achieves 0.079 regret vs. CH’s 0.207 (Table 3); on 8 hard real datasets (CH regret > 0.3), METAIVM wins all 8. The graph extension provides evidence beyond Euclidean data. Limited non-tabular evidence (5 text, 4 image) tempers cross-modality claims. Deployment cost is negligible: < 1 s feature extraction, < 0.2 ms inference, cheaper than Silhouette (Table 27, Appendix).

Limitations. (1) Requires offline training on labeled datasets (mitigated: can use purely synthetic data). (2) Predicts external agreement (AMI), one operationalization of quality; domain-specific criteria may differ. (3) Limited evidence on specialized domains (genomics, time series). (4) Main benchmark omits Spectral Clustering ($O(n^3)$); post-hoc inclusion increases regret by only 0.005. (5) Struggles on high-noise, subspace, and extreme-imbalance datasets (19% of test cases above regret 0.15). (6) The AutoClust-style baseline is a partial reproduction using four of the ten original CVIs; the other six (SDBw, CDBw, Tau, Ratkowsky-Lance, McClain-Rao, C-Index) lack maintained Python implementations. The effect of the unavailable six CVIs on the reproduction regret remains unknown.

8 Conclusion

On our benchmark, external-agreement-based clustering model selection is substantially better served by a learned surrogate than by any fixed IVM. METAIVM reduces selection regret by 67% over the strongest classical baseline (Calinski-Harabasz, 0.217), with the improvement over Silhouette, CH, and DB significant at Wilcoxon $p < 10^{-12}$; density-based measures (DBCV 0.219, S_Dbw 0.334) are comparable to but no better than CH on the same protocol. Improvements are robust across metrics and models and extend to graph community detection. Principled controls confirm that the gains come from the per-partition formulation and feature space, not from learning alone or the inclusion of IVMs. Performance depends on the training distribution, and generalization to domains far from our benchmark remains an open question. All data, code, and a ready-to-use model-selection interface are publicly available at <https://github.com/hsevi/metaivm>.

Ethics and reproducibility. We foresee no significant negative societal impact. The benchmark uses only publicly available datasets. All code, datasets, and experiment scripts are released as open-source. The benchmark is fully reproducible with fixed random seeds (0–4) for all splits.

References

- Olatz Arbelaiz, Ibai Gurrutxaga, Javier Muguerza, Jesús M Pérez, and Iñigo Perona. An extensive comparative study of cluster validity indices. *Pattern Recognition*, 46(1):243–256, 2013.
- Beatriz Flámia Azevedo, Ana Maria AC Rocha, and Ana I Pereira. A multi-objective clustering approach based on different clustering measures combinations: Bf azevedo et al. *Computational and Applied Mathematics*, 44(2):59, 2025.
- Anna Beer, Lena Krieger, Pascal Weber, Martin Ritzert, Ira Assent, and Claudia Plant. Internal evaluation of density-based clusterings with noise. In *International Conference on Learning Representations*, volume 2026, pp. 86149–86172, 2026.

- Bernd Bischl, Pascal Kerschke, Lars Kotthoff, Marius Lindauer, Yuri Malitsky, Alexandre Frechette, Holger Hoos, Frank Hutter, Kevin Leyton-Brown, Kevin Tierney, and Joaquin Vanschoren. ASlib: A benchmark library for algorithm selection. *Artificial Intelligence*, 237:41–58, 2016.
- Tadeusz Caliński and Jerzy Harabasz. A dendrite method for cluster analysis. *Communications in Statistics – Theory and Methods*, 3(1):1–27, 1974.
- Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- Matheus Camilo da Silva, Gabriel Marques Tavares, Eric Medvet, and Sylvio Barbon Junior. Problem-oriented AutoML in clustering. *arXiv preprint arXiv:2409.16218*, 2024.
- David L Davies and Donald W Bouldin. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(2):224–227, 1979.
- Daniel G Ferrari and Leandro N De Castro. Clustering algorithm recommendation: A meta-learning approach. *Knowledge and Information Systems*, 44(1):65–94, 2015.
- Santo Fortunato and Marc Barthélemy. Resolution limit in community detection. *Proceedings of the National Academy of Sciences*, 104(1):36–41, 2007.
- Maria Halkidi, Yannis Batistakis, and Michalis Vazirgiannis. Clustering validity checking methods: part ii. *ACM SIGMOD Record*, 31(3):19–27, 2002.
- Hyeon Jeon, Michaël Aupetit, DongHwa Shin, Aeri Cho, Seokhyeon Park, and Jinwook Seo. Measuring the validity of clustering validation datasets. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(6):5045–5058, 2025.
- Ken Lang. Newsweeder: Learning to filter netnews. In *Proceedings of the 12th International Conference on Machine Learning*, pp. 331–339, 1995.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Ana C. Lorena, Luís P. F. Garcia, Jens Lehmann, Marcilio C. P. de Souto, and Tin Kam Ho. How complex is your classification problem? a survey on measuring classification complexity. *ACM Computing Surveys*, 52(5):1–34, 2019.
- Davoud Moulavi, Pablo A. Jaskowiak, Ricardo J.G.B. Campello, Arthur Zimek, and Jörg Sander. Density-based clustering validation. In *SIAM International Conference on Data Mining*, pp. 839–847. SIAM, 2014.
- Mark EJ Newman. Modularity and community structure in networks. *Proceedings of the National Academy of Sciences*, 103(23):8577–8582, 2006.
- Bruno A Pimentel and André CPLF de Carvalho. A new data characterization for selecting clustering algorithms using meta-learning. *Information Sciences*, 477:203–219, 2019.
- Yannis Poulakis, Christos Doulkeridis, and Dimosthenis Kyriazis. AutoClust: A framework for automated clustering based on cluster validity indices. In *IEEE International Conference on Data Mining (ICDM)*, pp. 1220–1225. IEEE, 2020.
- Peter J Rousseeuw. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.
- Alasdair Simpson, Mario A Munoz, and Kate Smith-Miles. An instance space analysis of clustering validation measures. *Data Mining and Knowledge Discovery*, 2026.
- Kate Smith-Miles and Mario Andrés Muñoz. Instance space analysis for algorithm testing: Methodology and software tools. *ACM Computing Surveys*, 55(12):1–31, 2023.

Vincent A Traag, Rodrigo Aldecoa, and Jean-Charles Delvenne. Detecting communities using asymptotical surprise. *Physical Review E*, 92(2):022816, 2015.

Joaquin Vanschoren, Jan N van Rijn, Bernd Bischl, and Luis Torgo. OpenML: Networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60, 2014.

David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996.

Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

A Complete Feature Descriptions

Table 10 lists all 56 features extracted for each (dataset, partition) pair, grouped by source.

B Feature Importance

Table 11 shows the complete XGBoost feature importance ranking. The top 5 features account for 62% of total importance; the top 15 account for 84%.

Protocol-specific permutation-importance intervals. We report permutation importance with a bootstrap uncertainty interval that reflects variability across random permutation realizations at a single split, not variability across datasets or folds. The protocol is: (i) fix the seed-0 split of the 5-seed 60/20/20 dataset-level protocol (train 133 datasets, val 44 datasets, test 46 datasets); (ii) train the default 28-feature XGBoost once on train+val at that split; (iii) for each feature and each of 30 independent random permutations, shuffle that feature’s column across the test-fold rows and recompute the selection regret; (iv) construct a 95% bootstrap interval on the mean Δ regret across the 30 permutation runs by resampling those 30 realizations 1000 times with replacement (`numpy.random` with a fixed seed). Table 12 reports mean Δ regret and CI endpoints. Features whose lower CI bound remains strictly positive after rounding to four decimals are marked as clearing this protocol-specific threshold; the lower bound for `graph_normalized_cut_proxy` is +0.0002 at four-decimal precision (previously printed as +0.000 due to three-decimal rounding). Eight features clear the threshold: `graph_conductance_min`, `centroid_sep_mean`, `centroid_sep_min`, `noise_fraction`, `cluster_size_max`, `cluster_density_max`, `cluster_size_std`, and `graph_normalized_cut_proxy`. Per-feature impacts are modest in absolute magnitude because the 28 features are redundantly informative (removing any single feature still leaves correlated structural features that partially recover the signal). We do not apply a multiple-testing correction, and we do not aggregate the analysis across the other four seeds; the resulting significance statements should be read as protocol-specific rather than as full-benchmark tests of feature relevance.

C Clustering Algorithm Configurations

Table 13 details the complete hyperparameter grid for each clustering algorithm (91 configurations in total). At runtime, SpectralClustering was skipped due to $O(n^3)$ complexity, and AgglomerativeClustering was skipped for datasets with $n > 10,000$ due to $O(n^2)$ memory. After these filters, the effective count ranges from 19 to 79 per dataset (mean: 75.7; lower counts occur for the largest datasets where additional GMM configurations also fail to converge), yielding 16,889 total runs across 223 datasets.

D Per-Seed Results

Table 14 shows selection regret and Spearman ρ for each method on each of the 5 random seeds. Results are consistent across seeds, with low variance for METAIVM.

Table 10: Complete feature list. All features are computed without access to ground-truth labels.

Feature	Description
<i>Partition statistics (12 features)</i>	
<code>n_clusters</code>	Number of clusters in the partition
<code>noise_fraction</code>	Fraction of points assigned to noise (-1 label)
<code>cluster_size_{min,max,mean,std,median}</code>	Statistics of cluster sizes
<code>size_ratio</code>	Ratio of largest to smallest cluster
<code>entropy</code>	Shannon entropy of the cluster size distribution
<code>gini</code>	Gini impurity of cluster assignments
<code>imbalance</code>	Deviation from uniform cluster sizes
<code>singleton_fraction</code>	Fraction of clusters with a single point
<i>Dataset descriptors (14 features)</i>	
<code>ds_n_samples, ds_n_features</code>	Dataset dimensions
<code>ds_log_n, ds_log_d</code>	Log-transformed dimensions
<code>ds_d_over_n</code>	Dimensionality ratio d/n
<code>ds_mean_of_{means,stds,skewness,kurtosis}</code>	Feature-wise distribution statistics (averaged)
<code>ds_sparsity</code>	Fraction of near-zero entries
<code>ds_intrinsic_dim_pca95</code>	PCA components for 95% variance
<code>ds_pairwise_dist_{mean,std,median}</code>	Pairwise distance statistics (subsamped to 2k points)
<i>Partition-data interaction features (10 features)</i>	
<code>within_cluster_var_{mean,std}</code>	Within-cluster variance statistics
<code>between_cluster_var</code>	Between-cluster variance
<code>dispersion_ratio</code>	Between / within variance ratio
<code>cluster_density_{mean,std,max}</code>	Average pairwise distance within clusters
<code>centroid_sep_{min,mean,std}</code>	Pairwise centroid distance statistics
<i>Graph-based features (6 features)</i>	
<code>graph_cut_fraction</code>	Fraction of k NN edges crossing cluster boundaries
<code>graph_normalized_cut_proxy</code>	Proxy for the normalized graph cut
<code>graph_within_cluster_edge_density</code>	Mean edge density within clusters
<code>graph_within_edge_density_std</code>	Std of within-cluster edge density
<code>graph_conductance_{min,mean}</code>	Graph conductance statistics
<i>IVM features (3 features)</i>	
<code>ivm_silhouette</code>	Silhouette Coefficient
<code>ivm_calinski_harabasz</code>	Calinski-Harabasz Index
<code>ivm_davies_bouldin_neg</code>	Negated Davies-Bouldin Index
<i>Algorithm encoding (11 features)</i>	
<code>algo_{KMeans,...,Agglom.}</code>	One-hot encoding of algorithm family (6)
<code>hp_{n_clusters,...,min_cluster_size}</code>	Numeric hyperparameter values (5)

E Dataset Collection Details

Our benchmark comprises 223 datasets from four sources. Table 15 provides summary statistics.

OpenML datasets (91). We selected datasets from the OpenML CC18 benchmark suite (Bischl et al., 2016) plus 32 additional hand-selected datasets chosen for domain diversity (biology, finance, physics, social sciences, computer vision). Datasets with $n > 20,000$ samples were subsampled uniformly at random; those with $d > 200$ features were reduced to 50 dimensions via PCA. Three datasets (`openml_3`, `openml_46`, `openml_50`) were excluded due to all-NaN features after preprocessing.

Synthetic datasets (123). Generated using scikit-learn generators and custom code to systematically vary:

Table 11: Complete feature importance ranking (XGBoost gain). Features below 0.5% omitted for brevity.

Rank	Feature	Importance
1	ivm_calinski_harabasz	34.96%
2	ds_mean_of_kurtosis	9.75%
3	graph_conductance_min	7.96%
4	cluster_size_mean	4.64%
5	ds_sparsity	4.43%
6	entropy	4.31%
7	ds_pairwise_dist_std	4.08%
8	ds_mean_of_skewness	2.57%
9	ds_d_over_n	2.15%
10	between_cluster_var	1.81%
11	noise_fraction	1.72%
12	ds_n_features	1.62%
13	gini	1.61%
14	ivm_silhouette	1.59%
15	ds_n_samples	1.48%
16	singleton_fraction	1.43%
17	cluster_density_mean	1.18%
18	ds_pairwise_dist_median	1.08%
19	ds_mean_of_means	0.92%
20	algo_GaussianMixture	0.82%

Table 12: Permutation importance with 95% bootstrap CIs (30 permutations, default 28-feature model, seed-0 split of the 5-seed 60/20/20 protocol). Δ regret is the change in selection regret when the feature is shuffled at test time. Values shown to three decimals except the `graph_normalized_cut_proxy` row, whose lower bound is +0.0002 at four decimals (would round to +0.000 otherwise). * = lower 95% CI strictly > 0 . Top 20 features shown. See Section 6 for protocol details and scope caveats.

Feature	Δ regret	95% CI
<code>graph_conductance_min</code> *	+0.035	[+0.017, +0.050]
<code>centroid_sep_mean</code> *	+0.026	[+0.009, +0.043]
<code>centroid_sep_min</code> *	+0.025	[+0.005, +0.042]
<code>noise_fraction</code> *	+0.022	[+0.014, +0.031]
<code>cluster_size_max</code> *	+0.020	[+0.010, +0.031]
<code>entropy</code>	+0.015	[−0.000, +0.033]
<code>cluster_density_max</code> *	+0.013	[+0.002, +0.026]
<code>cluster_size_std</code> *	+0.012	[+0.004, +0.022]
<code>centroid_sep_std</code>	+0.010	[−0.005, +0.027]
<code>between_cluster_var</code>	+0.009	[−0.003, +0.023]
<code>within_cluster_var_std</code>	+0.009	[−0.001, +0.022]
<code>cluster_size_mean</code>	+0.009	[−0.005, +0.022]
<code>cluster_density_mean</code>	+0.009	[−0.001, +0.019]
<code>cluster_density_std</code>	+0.008	[−0.004, +0.018]
<code>graph_normalized_cut_proxy</code> *	+0.008	[+0.0002, +0.0150]
<code>graph_within_edge_density_std</code>	+0.007	[−0.003, +0.016]
<code>cluster_size_min</code>	+0.006	[−0.005, +0.018]
<code>within_cluster_var_mean</code>	+0.006	[−0.005, +0.015]
<code>cluster_size_median</code>	+0.006	[−0.001, +0.013]
<code>graph_conductance_mean</code>	+0.005	[−0.005, +0.017]

- **Dimensionality:** $d \in \{2, 3, 5, 10, 15, 20, 30, 50, 100, 200\}$
- **Number of clusters:** $k \in \{2, 3, 4, 5, 6, 8, 10, 12, 15, 20\}$

Table 13: Clustering algorithm hyperparameter grids. Total configurations: 91 per dataset (before filtering).

Algorithm	Hyperparameters	Configs
KMeans	$k \in \{2, 3, 5, 7, 10, 15, 20, 30\}$, $n_init = 10$	8
Gaussian Mixture	$k \in \{2, 3, 5, 7, 10, 15, 20\}$, $cov \in \{\text{full, diag}\}$	14
Spectral	$k \in \{2, 3, 5, 7, 10, 15\}$, $affinity \in \{\text{rbf, nn}\}$	12*
DBSCAN	$\epsilon \in \{0.1, 0.3, 0.5, 1.0, 2.0, 5.0\}$, $min_samp \in \{3, 5, 10, 20\}$	24
HDBSCAN	$min_cluster \in \{5, 10, 20, 50\}$, $min_samp \in \{1, 5, 10\}$	12
Agglomerative	$k \in \{2, 3, 5, 7, 10, 15, 20\}$, $link \in \{\text{ward, comp., avg.}\}$	21 [†]

*Skipped at runtime (cubic complexity). [†]Skipped for $n > 10,000$ (quadratic memory).

Table 14: Per-seed results for all methods (selection regret / Spearman ρ).

Method	Seed 0	Seed 1	Seed 2	Seed 3	Seed 4
Random	.170 / .035	.160 / .030	.212 / .006	.171 / .007	.174 / .019
Global best	.236 / .106	.159 / .061	.352 / .024	.327 / .010	.130 / .063
Silhouette	.292 / .375	.390 / .369	.390 / .478	.361 / .467	.276 / .476
Calinski-Harabasz	.195 / .589	.249 / .573	.270 / .634	.230 / .604	.142 / .621
Davies-Bouldin	.297 / .337	.393 / .329	.395 / .427	.365 / .455	.277 / .448
CH Adjusted	.229 / .513	.288 / .490	.282 / .555	.282 / .535	.207 / .559
Ridge	.093 / .617	.111 / .620	.109 / .710	.072 / .693	.055 / .669
MLP	.093 / .614	.096 / .640	.096 / .721	.056 / .734	.070 / .733
MetaIVM (XGB)	.080 / .703	.075 / .744	.074 / .808	.060 / .768	.064 / .718

- **Cluster overlap:** Low ($\sigma = 0.5$), medium ($\sigma = 1.5$), high ($\sigma = 3.0$)
- **Cluster shape:** Spherical (blobs), ellipsoidal (random covariance), manifold (Swiss roll, S-curve), non-convex (moons, circles)
- **Imbalance:** Mild (1:1.5:2.5), moderate (1:3:16), extreme (1:1.5:2.5:45), many-small (6 clusters with one dominant)
- **Noise:** 5%, 15%, 25% uniform background noise
- **Subspace structure:** Relevant features in $\{2, 3, 5\}$ out of $\{50, 100, 200\}$ total dimensions

Text datasets (5). Five subsets of the 20Newsgroups corpus: all categories, science, politics+religion, computers, and recreation. Text was converted to TF-IDF vectors (5,000-word vocabulary, English stop words removed) and reduced to 50 dimensions via truncated SVD. Subsampled to 2,000–5,000 documents.

Image datasets (4). MNIST and Fashion-MNIST digit/clothing images, each at 2,000 and 5,000 samples. Pixel features (784 dimensions) were reduced to 50 dimensions via PCA.

F Preprocessing Pipeline

Each dataset undergoes the following preprocessing before clustering:

1. **Missing value handling:** Columns with $>50\%$ missing values are dropped. Remaining missing values are imputed with column medians.
2. **Constant feature removal:** Features with zero variance are dropped.
3. **Standardization:** All features are standardized to zero mean, unit variance.
4. **Graph construction:** k -nearest-neighbor graphs are computed for $k \in \{10, 15, 20\}$ using Euclidean distance, stored as sparse adjacency matrices.

Table 15: Dataset collection summary statistics (across 223 datasets with clustering runs).

Statistic	Min	Median	Mean	Max
Samples (n)	150	1,200	4,438	48,842
Features (d)	2	14	33	500
Classes (k)	2	4	6.3	50
Runs per dataset	19	79	75.7	91

G Zero Feature Engineering Details

Table 16 compares approaches with varying levels of feature engineering. All results are preliminary (single architectures, no hyperparameter search).

Table 16: Impact of feature engineering. Even zero-engineering approaches (graph embeddings, GNN) substantially outperform all classical IVMs.

Method	Engineering	Regret $\downarrow$	Spearman $\rho \uparrow$
CH (strongest classical baseline)	n/a	.217 $\pm$.237	.604 $\pm$.320
DBCV (density-based baseline)	n/a	.219 $\pm$.259	–
XGBoost (graph spectral emb.)	none	.105 $\pm$.130	.667 $\pm$.290
GNN (end-to-end)	none	.131 $\pm$.193	.705 $\pm$.229
XGBoost (auto-features, 18)	minimal	.083 $\pm$.120	.743 $\pm$.270
MetaIVM (hand-crafted, 56)	heavy	.073 $\pm$.103	.758 $\pm$.302

The GNN operates directly on the k NN graph with partition labels as 32-dimensional one-hot node features (3-layer GIN, 128 hidden dims; see Appendix H for details). Despite no feature engineering, it achieves $\rho = 0.705$, above CH ($\rho = 0.604$, the strongest of the three standard IVMs for which ranking metrics were computed). Graph spectral embeddings (top-32 truncated-SVD components of the k NN adjacency matrix, mean/std-pooled per cluster and concatenated with partition statistics) fed to XGBoost achieve lower regret (0.105) but lower ρ (0.667). The 18 auto-features (cluster sizes, within/between variance, centroid distances) close most of the gap to the full model. Hand-crafted features provide a further 12% improvement, primarily through graph conductance and dataset-level descriptors (kurtosis, sparsity).

H Model Training Details

XGBoost configuration. We use the `xgboost` Python package with the following hyperparameter search: max depth $\in \{3, 5, 7\}$, learning rate $\in \{0.01, 0.05, 0.1\}$ (9 combinations total). Each configuration is trained with up to 500 estimators and early stopping (patience 20 rounds) on the validation set, using RMSE as the stopping criterion. The best configuration (lowest validation MSE) is then refitted on the combined train+val data using the optimal number of estimators from early stopping. Missing feature values (e.g., IVM scores for degenerate partitions) are handled natively by XGBoost’s sparse-aware split finding.

MLP configuration. Grid search over hidden layer architectures $\{(256, 128), (512, 256), (256, 128, 64), (512, 256, 128)\}$ and learning rates $\{10^{-3}, 3 \cdot 10^{-3}\}$ (8 configurations total), ReLU activations, Adam optimizer, max 200 iterations. Each configuration is trained on the dataset-level train fold only (not on train + val pooled); scikit-learn’s built-in `early_stopping` is disabled to avoid within-training-set random splitting that would mix candidate partitions from the same dataset across sklearn’s internal train / val. Outer arch/lr model selection uses the dataset-level held-out val fold. Features are standardized to zero mean and unit variance. Missing values are imputed with zeros before scaling. This preserves dataset-level train/val separation throughout all model selection and training decisions.

Ridge regression. L_2 -regularized linear regression with regularization strength α selected from $\{0.01, 0.1, 1.0, 10.0, 100.0\}$ via validation R^2 score. The best α is used to refit on the combined train+val data. Features are standardized as above.

GNN configuration. 3 GIN layers with 128 hidden dimensions, residual connections, batch normalization, 10% dropout. Partition labels encoded as 32-dimensional one-hot vectors (with hash collision for $k > 31$). Node features truncated/padded to 50 dimensions. Partition-aware pooling: mean within each cluster, then mean across cluster representations, concatenated with global mean pool. Adam optimizer ($\text{lr} = 10^{-3}$), batch size 32, max 80 epochs, early stopping patience 15.

I Baseline Reproductions: AutoClust-style and PoAC-style

Neither AutoClust (Poulakis et al., 2020) nor PoAC (da Silva et al., 2024) released public code with their paper. We therefore reimplemented each method from the description in the paper. Both reproductions use our 5-seed 60/20/20 dataset-level splits (Section 5) rather than the original evaluation protocols of the source papers, so the reported regret numbers reflect performance on our 223-dataset benchmark rather than on the smaller UCI subsets in the original evaluations. This section documents our reproduction protocol and its deviations from the original descriptions.

AutoClust-style baseline. The relevant component of AutoClust is the surrogate MLP that maps a candidate partition’s classical CVIs to predicted external agreement. From Poulakis et al. (2020) Figure 2 and Table I:

- Architecture: hidden layers $60 \rightarrow 30 \rightarrow 10$, ReLU activations, ADAM optimizer.
- Target: external agreement (Adjusted Rand Index; we use AMI here to match the rest of our benchmark. AMI and ARI are strongly correlated on our benchmark, Table 23).
- Input: a fixed vector of internal CVIs. The original paper uses ten: Silhouette, Dunn, C-Index, Calinski-Harabasz, Davies-Bouldin, SDbw, CDBw, Tau, Ratkowsky-Lance, McClain-Rao.

Our reproduction matches the architecture exactly. For the input CVIs, we use four that have maintained Python implementations and that we could compute cleanly for all 16,889 partitions: Silhouette, Calinski-Harabasz, and Davies-Bouldin (from the paper’s `runs_master.csv`, matching the standard sklearn definitions used throughout the paper) and Dunn Index (implemented from the standard definition: minimum single-linkage inter-cluster distance divided by maximum intra-cluster diameter). The remaining six CVIs (SDbw, CDBw, Tau, Ratkowsky-Lance, McClain-Rao, C-Index) lack maintained Python implementations, and we did not reimplement them from scratch, to keep the baseline faithful to the original method rather than to our own re-derivations. We therefore refer to this as an *AutoClust-style partial reproduction* throughout the paper, and do not claim it matches the original design exactly. The regret of a learned MLP on four CVIs is 0.195 ± 0.028 , a modest improvement over the best single classical IVM (CH, 0.217) but far short of METAIVM (0.071). Whether adding the missing six CVIs would substantially change this remains an open empirical question; we return to it in the Limitations of Section 7.

PoAC-style baseline. The relevant component of PoAC is the surrogate Random Forest that maps (dataset meta-features, CVIs of the candidate partition) to external agreement. From da Silva et al. (2024) Section 5:

- Model class: Random Forest regressor.
- Dataset meta-features: the taxonomy of Lorena et al. (2019) covering statistical, information-theoretic, complexity, and concept groups.
- CVIs: Silhouette and Davies-Bouldin.

Table 17: Handling of degenerate partitions across CVI baselines. “Excluded” means the baseline cannot score the partition and receives a worst-case sentinel value during selection. Counts are computed over all 16,889 (dataset, partition) pairs in our benchmark; see the DBCV / S_Dbw availability CSV (`density_cvis.csv`) for the full per-partition status.

Condition	N	Sil	CH	DB	DBCV	S_Dbw	MetaIVM
Trivial ($k = 1$ after noise)	1,515	excl.	excl.	excl.	excl.	excl.	scored
All-noise ($> 95\%$ label = -1)	1,969	excl.	excl.	excl.	excl.	excl.	scored
Singleton-heavy ($> 50\%$ singletons)	1	excl.	scored	scored	scored	scored	scored
Noise pts ($= -1$) present	(mixed)	excl.	excl.	excl.	native	excl.	feature
Feasible (“ok”) partitions	13,404	scored	scored	scored	scored*	scored	scored

*DBCV computed on 11,609 of 13,404 feasible partitions (86.6%); of the remaining 1,795, 1,791 triggered numerical errors in `hdbscan.validity_index` and 4 lacked sufficient metadata (all excluded from DBCV selection). In the “Noise pts present” row, “excl.” means noise points are dropped before scoring, “native” means the CVI handles them as its own category, and “feature” means the noise fraction is available to MetaIVM as an input feature.

- Target: Adjusted Rand Index (we use AMI here for consistency).

Our reproduction matches: we use `scikit-learn’s RandomForestRegressor(n_estimators=100, random_state=0)`; the Lorena-2019 meta-features are extracted via the `pymfe` Python library (which is the maintained reference implementation of this taxonomy) with the same feature groups (`general`, `statistical`, `info-theory`, `concept`, `complexity`) and default `mean + sd` summary statistics. This yields 115 numerical feature columns per dataset (each base descriptor summarized as mean and standard deviation across instances or attributes; PyMFE returns 115 feature columns after removing constant/invalid summaries, plus one dataset identifier column). Combined with the two CVIs (Silhouette, Davies-Bouldin), the final PoAC-style training matrix has 117 features per (dataset, partition) pair. Silhouette and Davies-Bouldin are the standard sklearn implementations.

The one substantive deviation is the training partition source. The original PoAC paper synthesizes training partitions by adding Gaussian noise to ground-truth labels one hundred times per dataset, producing a controlled range of ARI values in the training set. Our benchmark contains 16,889 partitions from real clustering algorithms (KMeans, GMM, DBSCAN, HDBSCAN, Agglomerative) at real hyperparameter settings; we train PoAC on these instead. This matches our deployment candidate distribution more closely (PoAC-style scores real algorithm outputs at test time, not noise-augmented labels), but because it differs from PoAC’s original noise-augmentation protocol we do not claim that this deviation necessarily helps or hurts the reproduction; we present the result as a PoAC-style reproduction under our protocol rather than a strict replication. Under this protocol, PoAC-style achieves regret 0.139 ± 0.025 across our 5 seeds, well above MetaIVM (0.071) but below all classical IVM baselines.

Alignment with the paper’s control ladder. The B1 (Ridge on 3 IVMs, regret 0.337) and B2 (XGBoost on 3 IVMs + 14 dataset descriptors, regret 0.143) controls in Table 4 are structurally close to AutoClust’s and PoAC’s core designs but with fewer features. Both B1/B2 controls and the full AutoClust/PoAC reproductions land above MetaIVM’s 0.071 on our benchmark. Together they empirically bound the performance of learned surrogates that rely on classical CVIs (learned combinations of them, plus dataset-level meta-features) at approximately 0.13–0.34 regret on our benchmark, versus MetaIVM’s 0.071 which uses partition-structural features.

Handling of degenerate partitions across baselines. Classical CVIs are not defined for all partitions. Table 17 documents the invalidity conditions, counts on our benchmark, and per-baseline policy. When a CVI is undefined for a given partition, that baseline receives a worst-case score for the purposes of selection regret (i.e. the baseline is effectively told the partition is unusable), so the regret numbers in Table 2 reflect a defensible best-effort policy rather than a silent drop.

MetaIVM does not have per-partition invalidity conditions: its 28 features are always computable (`n_clusters=1` sets partition-only features to sentinel values but the graph and interaction features remain well-defined). This is one reason MetaIVM’s coverage is 100% across the benchmark while classical CVIs miss at least 3,484 of 16,889 partitions (20.6%) due to trivial or all-noise conditions.

J Evaluation Protocol

Dataset-level splits. For each random seed $s \in \{0, 1, 2, 3, 4\}$, we randomly partition the 223 datasets into 60% train (133 datasets), 20% validation (44 datasets), and 20% test (46 datasets). All clustering runs for a given dataset are assigned to the same split, preventing any information leakage between train and test. Across the 5 seeds, the union of test folds covers 158 unique datasets; the remaining 65 datasets do not appear in any test fold under this split configuration and appear only in training or validation roles.

Metric definitions.

- **Selection regret:** $\text{Regret}(d) = \max_j \text{AMI}_j - \text{AMI}_{j^*}$ where $j^* = \arg \max_j \hat{q}_j$. Averaged across test datasets, then across seeds.
- **ε -success:** Fraction of test datasets satisfying $\text{Regret}(d) \leq 0.01$, i.e., where the selected partition is within an absolute AMI gap of 0.01 from the best available partition.
- **NDCG@5:** Normalized discounted cumulative gain using true AMI as relevance, evaluated on the top 5 predicted partitions.
- **Spearman ρ :** Rank correlation between predicted scores and true AMI, computed per dataset, then averaged.
- **Kendall τ :** Concordance probability for all pairwise comparisons within a dataset, then averaged.

IVM baselines. Classical IVMs require no training. For each test dataset, the partition with the highest IVM score (or lowest, for Davies-Bouldin) is selected. The adjusted CH index (Jeon et al., 2025) is computed using the official `btwim` Python library with 20 Monte Carlo iterations for the permutation-based normalization, applied pairwise across all cluster pairs. We follow the default setting of 20 iterations from Jeon et al.; spot checks with 50 iterations showed $<0.5\%$ change in mean CH_A values, indicating adequate convergence for model selection purposes.

K Constructive Illustration: A CH Failure Mode

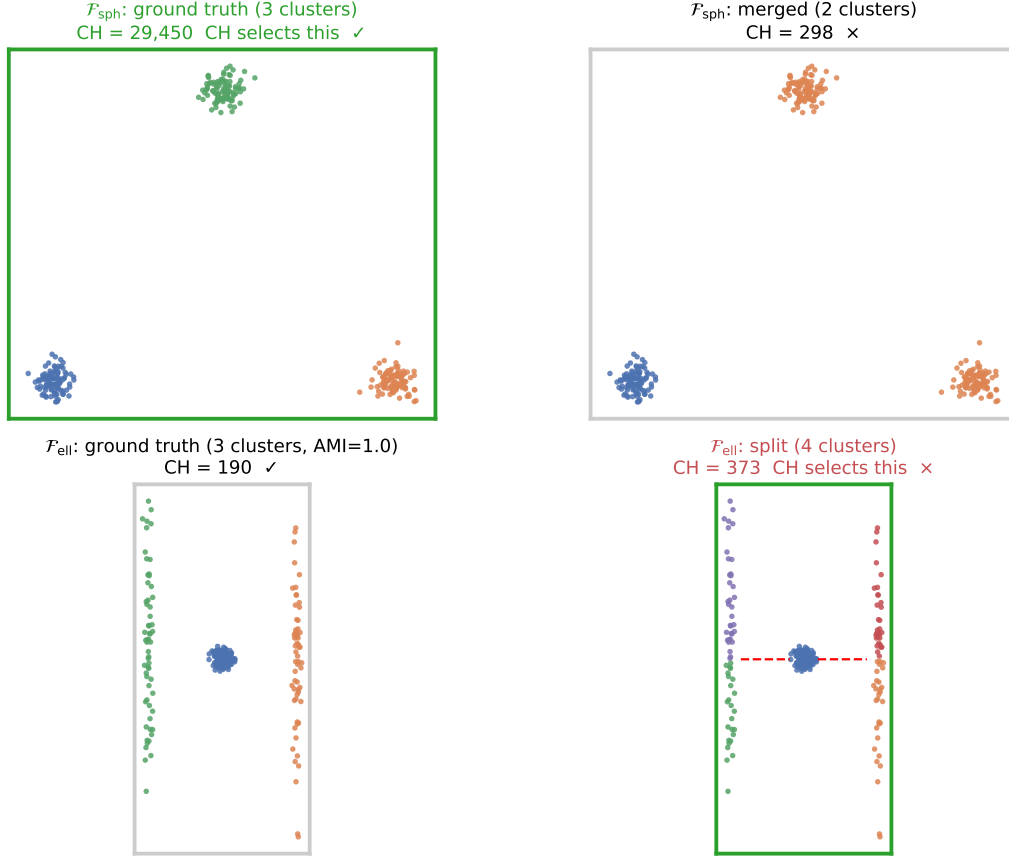
This appendix presents a small constructive illustration of a concrete failure mode of the Calinski-Harabasz index (CH). It is not a general theorem: we do not attempt to characterize the class of all IVMs for which the failure mode holds, and we do not claim the construction rules out other fixed IVMs universally. What we do show is that on the constructed pair of dataset families, CH is actively misled in a mechanically predictable way (splitting elongated clusters reduces its within-cluster dispersion component), and that a learned evaluator with access to partition-structural features avoids this failure empirically on the full benchmark (Section 6, Table 2). This is an illustrative instance of assumption (A3) failing for the specific feature map used by CH; the negative- ρ failure rates we report at the end of the section quantify how often the failure mode manifests on real data.

The construction. We define two dataset families and, on each, a pair of candidate partitions π_a, π_b . Throughout, let

$$W(\mathcal{X}, \pi) = \frac{1}{n} \sum_{j=1}^k \sum_{i: \pi(i)=j} \|x_i - \mu_j\|^2, \quad B(\mathcal{X}, \pi) = \frac{1}{n} \sum_{j=1}^k |C_j| \cdot \|\mu_j - \bar{x}\|^2,$$

with μ_j the centroid of cluster j and $\bar{x}$ the global mean.

Constructive illustration of a CH failure mode



Top row: CH correctly prefers the ground truth. Bottom row: CH prefers the variance-reducing split ($373 > 190$), disagreeing with the ground truth.

Figure 2: Constructive CH failure mode; each row shows a dataset family under two candidate partitions, with the CH-selected partition (higher CH) outlined in green. **Top ($\mathcal{F}_{\text{sph}}$, isotropic Gaussians):** CH correctly prefers the ground-truth 3-cluster partition ($\text{CH}=29,450$) over the merged 2-cluster partition ($\text{CH}=298$). **Bottom ($\mathcal{F}_{\text{ell}}$, heterogeneous covariance):** CH incorrectly prefers the partition that splits the elongated clusters ($\text{CH}=373$, dashed red lines) over the ground truth ($\text{CH}=190$, $\text{AMI}=1.0$), because splitting reduces within-cluster variance. The failure is that maximizing CH selects the wrong partition in the bottom row.

Family $\mathcal{F}_{\text{sph}}$. Let $\mathcal{X}_1 \subset \mathbb{R}^2$ consist of $n = 300$ points drawn from three isotropic Gaussians with $\sigma = 0.3$ and centers $c_1 = (0, 0)$, $c_2 = (10, 0)$, $c_3 = (5, 8.66)$ (equilateral triangle, side length 10). Consider:

- π_a : the correct 3-cluster assignment (each point to its generating center);
- π_b : a 2-cluster assignment that merges clusters 2 and 3.

For this dataset the three isotropic Gaussians are well-separated, so π_a has lower W and higher B than π_b . Empirically (Figure 2 top row, computed with 100 points per Gaussian and $\sigma = 0.3$) $\text{CH}(\mathcal{X}_1, \pi_a) = 29,450 > \text{CH}(\mathcal{X}_1, \pi_b) = 298$, which agrees with $\text{AMI}(\pi_a, y^*) > \text{AMI}(\pi_b, y^*)$.

Family $\mathcal{F}_{\text{ell}}$. Let $\mathcal{X}_2 \subset \mathbb{R}^2$ consist of three groups:

- Cluster 1: $n_1 = 400$ points from $\mathcal{N}((0, 0), 0.2^2 I)$, a dense, compact sphere;

- Cluster 2: $n_2 = 100$ points from $\mathcal{N}((3, 0), \text{diag}(0.1^2, 2.5^2))$, a vertically elongated ellipsoid;
- Cluster 3: $n_3 = 100$ points from $\mathcal{N}((-3, 0), \text{diag}(0.1^2, 2.5^2))$, another vertically elongated ellipsoid.

Consider:

- π_a : a 4-cluster partition that keeps the dense sphere as one cluster but splits each ellipsoid horizontally at $y = 0$;
- π_b : the correct 3-cluster assignment matching the generating distribution.

The four-cluster split π_a reduces W (each half-ellipsoid is more compact than the full ellipsoid) and increases B (the half-ellipsoid centroids are further apart vertically). Empirically (Figure 2 bottom row, computed with 200 points in the dense sphere and 60 points in each elongated ellipsoid) $\text{CH}(\mathcal{X}_2, \pi_a) = 373 > \text{CH}(\mathcal{X}_2, \pi_b) = 190$, but $\text{AMI}(\pi_b, y^*) = 1.0 > \text{AMI}(\pi_a, y^*)$. CH prefers the split that reduces variance even though the split violates the underlying generating structure.

What this demonstrates. On the two constructed families, CH ranks the correct partition first on $\mathcal{F}_{\text{sph}}$ but last on $\mathcal{F}_{\text{ell}}$, and the mismatch is driven by CH’s dependence on B/W : splitting an elongated cluster along its major axis always reduces W regardless of whether the split is semantically meaningful. This is the specific empirical signal that partition-structural features (elongation of within-cluster covariance, cluster-size imbalance, graph conductance on the k NN graph) are designed to capture, and that CH’s B/W summary cannot. We do not extend this argument to other fixed IVMs (Davies-Bouldin, Silhouette, DBCV, S_Dbw depend on per-cluster or density-based quantities not captured by the B/W ratio, and their failure modes are qualitatively different from CH’s, though the benchmark results in Table 2 show that all of them are outperformed by METAIVM in aggregate).

Empirical verification on the benchmark. On our benchmark (averaged across 5 seeds), CH achieves positive Spearman ρ on 94% of test datasets but *negative* ρ on 6%, meaning that on those 6% CH actively prefers worse partitions in the ordering sense. METAIVM achieves positive ρ on 98% and negative on only 2%. The 6% negative-correlation rate for CH corresponds to real datasets with non-spherical structure or imbalanced cluster sizes, consistent with the failure mode illustrated above.

L Compact Feature Set: Selection Protocol

The 15-feature compact model is constructed as follows:

1. For each of the 5 training seeds, train XGBoost on the training fold and extract feature importances (gain-based).
2. Rank features by importance for each seed.
3. Select the 15 features that appear in the top-15 for at least 3 out of 5 seeds.

The resulting 15 features are: `ivm_calinski_harabasz`, `ds_mean_of_kurtosis`, `graph_conductance_min`, `entropy`, `ds_mean_of_skewness`, `ds_n_samples`, `ds_pairwise_dist_std`, `cluster_size_mean`, `graph_conductance_mean`, `ds_pairwise_dist_mean`, `between_cluster_var`, `ds_pairwise_dist_median`, `ivm_silhouette`, `ds_d_over_n`, `noise_fraction`.

Eight of these features are stable across *all* 5 seeds: `ivm_calinski_harabasz`, `ds_mean_of_kurtosis`, `graph_conductance_min`, `entropy`, `ds_mean_of_skewness`, `ds_n_samples`, `ds_pairwise_dist_std`, `cluster_size_mean`. Importantly, feature selection is performed exclusively on training folds; test data is never used for selection.

M Additional Experimental Tables

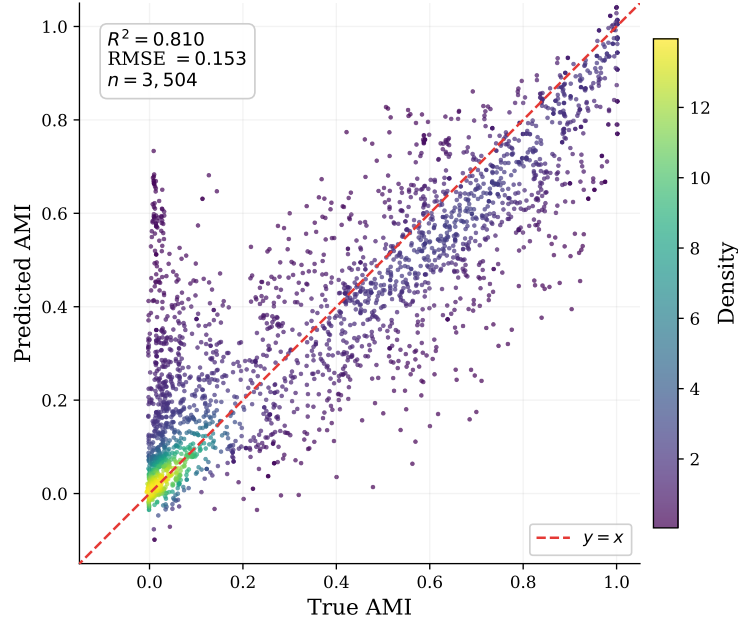


Figure 3: Predicted vs. true AMI for all (dataset, partition) pairs on the seed-0 test fold ($R^2 = 0.810$, $\text{RMSE} = 0.153$, $n = 3,504$: 46 test datasets $\times$ mean 76.2 partitions per dataset). Points near the diagonal indicate accurate prediction. The model captures the full range of external agreement, from poor ($\text{AMI} \approx 0$) to excellent ($\text{AMI} \approx 1$). Full 5-seed regret and correlation aggregates are in Table 2 and Table 20.

Table 18: Cross-domain transfer. METAIVM trained on one domain, tested on another. CH shown as reference (the strongest of the three standard compactness-separation IVMs by regret in the in-distribution benchmark).

Scenario	MetaIVM		CH (reference)	
	Regret $\downarrow$	$\rho \uparrow$	Regret $\downarrow$	$\rho \uparrow$
In-distribution (60/20/20)	.071	.748	.217	.604
Synthetic $\rightarrow$ real	.100	.678	.235	.489
Synthetic $\rightarrow$ OpenML	.100	.657	.218	.466
Synthetic $\rightarrow$ image+text	.105	.887	.398	.722
Real $\rightarrow$ synthetic	.238	.607	.193	.761
OpenML $\rightarrow$ synthetic	.209	.590	.193	.761

Table 19: Leave-one-algorithm-out: impact of removing each algorithm family from training.

Held-out Algorithm	Regret (without)	Δ Regret
KMeans	.047 $\pm$.084	+ .004
Gaussian Mixture	.047 $\pm$.075	+ .004
DBSCAN	.043 $\pm$.107	+ .023
HDBSCAN	.022 $\pm$.044	- .001
Agglomerative	.049 $\pm$.065	+ .004

Table 20: Per-dataset Spearman correlation between predicted scores and true AMI (averaged over the 230 seed–dataset test evaluations of the 5-seed 60/20/20 protocol; 158 unique datasets appear in at least one test fold).

Method	Spearman ρ
METAIVM (predicted)	0.748 ± 0.259
CH Adjusted (Jeon et al., 2025)	0.531 ± 0.281
Calinski-Harabasz	0.604 ± 0.320
Silhouette	0.433 ± 0.402
Davies-Bouldin (neg)	0.399 ± 0.385

Table 21: Preliminary model selection on 7 real-world attributed graphs (189 candidate partitions); regret uses NMI as the external agreement metric. We report a single protocol: 7-fold leave-one-graph-out cross-validation (each fold trains on 6 graphs, tests on the held-out 7th, regret averaged over folds). With only 7 graphs the estimate is high-variance, so we present it as a preliminary indication rather than a headline result. Classical IVMs use only node attributes (Silhouette and CH are anti-correlated with quality); modularity uses only graph structure. Datasets (Cora, CiteSeer, PubMed, Amazon Photo/Computers, Coauthor CS/Physics) are loaded via `torch_geometric` and require download.

Method	Regret $\downarrow$	Spearman $\rho \uparrow$
<i>Attribute-only baselines (ignore graph structure)</i>		
Silhouette (on node attributes)	.395	−.588
Calinski-Harabasz (on node attributes)	.350	−.481
Davies-Bouldin (on node attributes)	.039	.548
<i>Graph-only baseline (ignore node attributes)</i>		
Modularity	.028	.592
MetaIVM (graph features, 7-fold leave-one-graph-out)	.004	.657

Table 22: Training objective comparison under the paper’s exact 5-seed 60/20/20 protocol. All three objectives use the same XGBoost architecture and hyperparameters. Differences are modest relative to the gap over classical IVMs (CH regret ≥ 0.20).

Training Objective	Regret $\downarrow$	Spearman $\rho \uparrow$
Pointwise regression (MSE)	$.068 \pm .012$	$.761 \pm .033$
LambdaMART (rank:ndcg)	$.062 \pm .017$	$.782 \pm .021$
Pairwise (rank:pairwise)	$.065 \pm .015$	$.796 \pm .027$
<i>CH (strongest CS-IVM by ρ)</i>	$.217 \pm .237$	$.604 \pm .320$

Table 23: Selection regret across four external quality metrics (5 seeds, 28-feature set). The AMI regret here (0.076) differs slightly from Table 2 (0.071) because each metric uses a separately trained model and the averaging scope differs.

Method	AMI	ARI	NMI	V-measure
METAIVM (XGBoost)	.076	.092	.072	.072
Ridge	.123	.152	.125	.125
Calinski-Harabasz	.217	.250	.219	.219
Silhouette	.342	.380	.340	.340
Davies-Bouldin	.345	.374	.345	.345

Table 24: Paired Wilcoxon signed-rank tests: METAIVM (XGBoost) vs. each baseline. Per-dataset regret averaged across 5 seeds, 158 test datasets.

Comparison	XGB wins	Baseline wins	Ties	p -value
vs. Calinski-Harabasz	110	39	9	4.1×10^{-13}
vs. Silhouette	139	13	6	1.5×10^{-24}
vs. Davies-Bouldin	137	15	6	2.4×10^{-24}
vs. Ridge	74	69	15	2.2×10^{-2}

Table 25: Learning curves: regret and Spearman ρ vs. training set size (fraction of 133 datasets, 5 seeds).

Frac.	N_{train}	XGBoost		Ridge	
		Regret $\downarrow$	$\rho \uparrow$	Regret $\downarrow$	$\rho \uparrow$
10%	13	.116	.717	.108	.622
20%	26	.117	.696	.108	.624
30%	39	.085	.739	.107	.625
50%	66	.083	.758	.110	.629
70%	93	.085	.759	.126	.625
100%	133	.076	.760	.123	.634

Table 26: Top failure cases: datasets with highest average selection regret for METAIVM (XGBoost), across 5 seeds.

Dataset	Regret	Best AMI	Failure mode
synth_uniform_noise_2d	0.658	0.989	High noise, ambiguous structure
synth_subspace_5of50	0.575	0.996	Subspace clustering
synth_imb_many_small_d15	0.429	1.000	Extreme imbalance
synth_blobs_many_2d	0.379	0.992	Many clusters, 2D
openml_1040 (sylva_prior)	0.367	0.478	High-d (108), low signal
synth_moons_noise0.15	0.327	0.818	Non-convex + noise
synth_mixed_shapes_2d	0.309	1.000	Mixed cluster shapes

Table 27: Asymptotic complexity per dataset with n samples, d features, k clusters, and M candidate partitions. METAIVM is cheaper than Silhouette and comparable to CH/DB.

Component	Complexity	Note
<i>METAIVM (deployment)</i>		
k NN graph	$O(n \log n \cdot d)$	One-time per dataset, reusable
Feature extraction	$O(M \cdot n \cdot k)$	Per partition: cluster stats + graph features
Model inference	$O(M \cdot T \cdot d_f)$	T =trees, d_f =28 features; < 1 ms
Total	$O(n \log n \cdot d + M \cdot n \cdot k)$	
<i>Classical IVMs</i>		
Silhouette	$O(M \cdot n^2)$	Pairwise distance matrix per partition
Calinski-Harabasz	$O(M \cdot n \cdot k)$	Comparable to METAIVM
Davies-Bouldin	$O(M \cdot n \cdot k)$	Comparable to METAIVM

Table 28: Candidate-pool robustness. METAIVM trained on the full pool, tested on modified pools at test time. METAIVM outperforms CH across all 11 configurations, supporting the view that the learned surrogate is not specialized to a particular candidate menu.

Pool configuration	METAIVM regret ↓	CH regret
Full pool (all algorithms)	.057	.199
Drop KMeans	.053	.205
Drop GaussianMixture	.048	.183
Drop DBSCAN	.055	.134
Drop HDBSCAN	.059	.191
Drop AgglomerativeClustering	.054	.195
KMeans only	.036	.085
Density-based only (DBSCAN+HDBSCAN)	.023	.119
Random 50% subsample	.055	.174
Only $k \geq 5$	.054	.139
Only $k \leq 3$	.042	.127